なぜ今 RSA 暗号は安全か？
量子コンピュータでどうやって破るのか

甘利 丈慈（Jorge Amari）

v1.0.0　2026 年 4 月

目次

第 1 章　はじめに 5

1.1　本書の使い方 5

1.2　本書の構成 7

第 2 章　RSA の安全性と RSA 暗号の原理 9

2.1　はじめに 9

2.2　RSA が破られた場合の影響 9

2.3　RSA 暗号を説明するための前提条件 14

2.4　RSA 暗号の手順を理解するための数学 16

2.5　RSA 暗号の手順 20

2.6　RSA の安全性 — 計算量の視点 23

2.7　まとめ 25

第 3 章　Shor のアルゴリズム 27

3.1　はじめに 27

3.2　位数発見問題（Order Finding Problem） 28

3.3　量子位相推定 31

3.4　Shor のアルゴリズム 38

3.5　実用性の分析 54

3.6　まとめ 80

3.7　参考文献 81

第 1 章

はじめに

「量子コンピュータが完成すれば RSA 暗号は破られる」——このフレーズを聞いたことがない人はいないだろう。2019 年の Google の「量子超越性」発表は、量子コンピューティングへの注目を押し上げた象徴的出来事であり、その後も研究進展、投資拡大、ポスト量子暗号への移行議論を背景に、社会的注目は依然として高い水準にある。そして、量子コンピュータのインパクトの大きい応用例の一つとして、RSA 暗号を破ることが注目されている。

しかし量子系の話題は、ときに過度に誇張され、量子コンピュータがあたかも魔法のように語られることがある。センセーショナルな見出しではなく内容を正しく理解したうえで、いま何ができて、何ができないのかを自分の手で確認できる状態を目指すべきである。

そこで本書では、RSA 暗号が現在安全とされている前提を押さえ、Shor のオリジナルアルゴリズム（Shor 1994）による素因数分解について、理論と体験（ハンズオン教材）で理解を深めていく。さらに、なぜ今の量子コンピュータではまだ RSA を破れないのかについても述べる。

1.1 本書の使い方

1.1.1 本書の特徴

本書では数式や導出を極力省略せず、各ステップで何が起きているのかを追えるように書いている。原理から理解することで、誇張なく数式を用いた論理展開で語れるようになることを目指す。

本書で扱うのは、Shor (1994) のオリジナルアルゴリズムである。将来 FTQC（Fault-Tolerant Quantum Computing; 誤り耐性量子計算）が実現した際には、目的やハードウェア特性に応じて Shor 系アルゴリズムの最適化版が使い分けられると考えられる（具体的な変種は 1.1.4 節を参照）。しかし、これらの最適化版も量子位相推定という基本原理を共有しており、オリジナルの Shor アルゴリズムは全ての基本形として、汎用的な標準実装の位置づけにある。オリジナルを理解することで、将来どの最適化版が使われても対応できる土台が得られる。

本書を読み、ハンズオンを体験することで、以下を理解できることを期待している。

- RSA 暗号はどのような仕組みで動いているのか
- なぜ古典コンピュータでは破れないのか
- 量子コンピュータはどのような手順で RSA を破るのか
- 古典と比べてどれくらい速いのか

- なぜ今の量子コンピュータではまだ破れないのか

1.1.2　前提知識

本書は第 2 章（RSA）と第 3 章（Shor）、および付属ハンズオンで構成されており（詳細は第 1.2 節）、前提知識は表 1.1 のとおりである。第 2 章は高校数学（合同式・指数）を前提とし、必要な整数論（オイラー関数等）は本文で導入する。第 3 章の Shor のアルゴリズムでは量子計算を取り扱うが、量子力学の事前知識がなくても読み進められるように構成している。ただし、線形代数（固有値問題を含む）と複素指数関数には慣れている前提とする。

なお、量子力学や量子コンピュータそのものへの理解は本書のスコープ外である。より深く学びたい場合は、量子力学や量子コンピュータに関する参考書を参照されたい。

補足として、読者の前提知識や本書から得たい知見は人それぞれであるため、自分の理解のために積極的に AI を活用してほしい。読者自身のレベルに沿って解説してくれるし、本書の内容を起点に、本書ではカバーできていない部分も検討・整理することができるだろう。

対象	必要な前提知識	補助
第 2 章（RSA）	高校数学（合同式・指数）。大学初年級レベルの整数論（オイラー関数等）は本文で導入	なし
第 3 章（Shor）	線形代数（固有値問題を含む）、複素指数関数	なし
付属ハンズオン	Python 基礎、Qiskit 初歩（量子レジスタ・測定・ビット順・シミュレータの感覚）	なし

表 1.1　本書・付属ハンズオンの前提知識

1.1.3　様々な人の使い方

本書は、量子コンピュータで RSA を破る基本的な話を、体系的に理解できるようにまとめたものである。

- **量子に興味がある人**
  - 最もインパクトのある RSA を破る応用例を通じて、量子コンピュータの理解を深められる
- **物理の学生**
  - RSA 暗号を破るという実問題の文脈で、QPE（量子位相推定）や QFT（量子フーリエ変換）に腰を据えて取り組める
  - ハンズオンで Qiskit を扱う練習にもなる
- **セキュリティエンジニア**
  - RSA がどう破られるのか？ 現状どれくらい危険なのかを理解したうえで、「PQC に移行しましょう」と提案できる
- **暗号に興味があるエンジニア**
  - RSA の数学的基盤を理解できる
  - RSA 問題・素因数分解・実運用上のパディング（OAEP/PSS）の関係を切り分けて理解できる

1.1.4　本書の範囲外

以下のトピックは本書では扱わない。

- **離散対数問題への Shor のアルゴリズム**
  Shor (1994) は素因数分解と離散対数問題の両方を解くアルゴリズムを提案したが、本書では素因数分解のみを扱う
- **Shor のアルゴリズムの変種**
  Ekerå–Håstad 等、試行回数と回路深度のトレードオフを改善した最適化版
- **他の量子アルゴリズム**
  Grover のアルゴリズム（探索）、量子機械学習等
- **他の暗号への量子脅威**
  楕円曲線暗号（ECDSA、ECDH）等
- **耐量子暗号（PQC）の詳細**
  格子暗号、ハッシュベース署名等の具体的な仕組み
- **量子誤り訂正の詳細**
  表面符号、論理量子ビットの具体的な実装
- **ハードウェア実装の詳細**
  超伝導、イオントラップ等の物理実装

1.2　本書の構成

本書は第 2 章と第 3 章で RSA と Shor のアルゴリズムを扱い、付属ハンズオン（Jupyter Notebook）で実装を体験する構成となっている。以下で各部分の概要を説明する。

1.2.1　第 2 章：RSA の安全性と RSA 暗号の原理

RSA 暗号の仕組みを解説し、その安全性が「素因数分解の計算量的困難さ」に依存していることを示す。具体的には、正当な受信者（Bob）の操作はすべて多項式時間で行えるのに対し、盗聴者（Eve）が解読するには素因数分解を経由する必要があり、これが計算量上の壁となっていることを述べる。

1.2.2　第 3 章：Shor のアルゴリズム

Shor のアルゴリズムの詳細を解説し、量子コンピュータが素因数分解を多項式時間で解ける原理と、なぜ今の量子コンピュータではまだ破れないのか（現状の技術的制約）を示す。

1. **位数発見問題**：素因数分解を位数発見に帰着させる
2. **量子位相推定（QPE）**：ユニタリ演算子の位相を読み取る汎用アルゴリズムを解説する
3. **Shor のアルゴリズムの手順**：量子位相推定の特別なケースで位数を発見する
4. **実用性の分析**：計算量の優位性、ノイズの影響、FTQC の実現、PQC への移行について述べる

1.2.3　付属ハンズオン（Colab 上で動作）

$N = 15$ の素因数分解（$15 = 3 \times 5$）を題材に、Shor のアルゴリズムを実際にコードを動かしながら体験する。まず古典計算で位数と因数分解の関係を確認し、古典計算の限界を体感する。次に、Qiskit を用いて加算器から段階的に量子回路を組み上げ、シミュレータ上で実行して因数分解を完了する。さらに、ノイズモデルを適用したシミュレーションにより、理想的な結果と現実のギャップを確認する。

第 2 章

RSA の安全性と RSA 暗号の原理

2.1　はじめに

RSA 暗号は、1977 年に Rivest、Shamir、Adleman によって発明された公開鍵暗号の代表格である。インターネット上の HTTPS 通信、デジタル署名、VPN など、現代の情報セキュリティの基盤を支えている。

本書の主役である Shor アルゴリズムは、この RSA の安全性の根拠とされる「素因数分解の困難さ」を突破する。Shor アルゴリズムの意義を正しく理解するためには、まず RSA がなぜ安全とされているのかを知る必要がある。

本章では、RSA 暗号の仕組みを解説し、その安全性が素因数分解の困難さとどのように結びついているかを示す（ただし、RSA 問題と素因数分解の完全な等価性は未証明である。詳しくは 2.6.3 節で述べる）。

2.2　RSA が破られた場合の影響

Shor アルゴリズムの意義を理解するために、まず「RSA が破られると何が起きるのか」を整理する。

2.2.1　RSA の用途：署名と暗号化

RSA は現代のインターネットセキュリティの至るところで使われている。その用途は大きく「署名」と「暗号化」の 2 つに分けられる。

表 2.1　RSA の主な用途

用途	目的	鍵の使い方
署名	認証（本人確認）	秘密鍵で署名、公開鍵で検証
暗号化	機密性（秘匿）	公開鍵で暗号化、秘密鍵で復号

RSA が現役で使われている場面

RSA 暗号化（公開鍵で暗号化、秘密鍵で復号）は以下の場面で現役である。

表 2.2　RSA 暗号化が現役で使われている場面

用途	具体例	備考
メール暗号化	S/MIME	企業のセキュアメールで広く利用
メール/ファイル暗号化	PGP/GPG	機密ファイルおよびメール（OpenPGP）の暗号化
鍵管理システム	HSM、KMS	鍵輸送・鍵カプセル化用途（限定的；デフォルトは AES Key Wrap、RSA は BYOK 等の特定用途）

これらの場面では、RSA は共通鍵を安全に届ける**鍵輸送**（key transport）として使われ、メッセージ本文は対称鍵暗号（AES 等）で暗号化する方式が主流である[*1]。具体的な流れは以下のとおりである。

1. **Alice が共通鍵 K を生成**（例：AES-256 用の 32 バイト鍵）
2. **Bob の RSA 公開鍵で K を暗号化** → 暗号文 C_{RSA}（鍵輸送）
3. **メッセージ本文は K で暗号化** → 暗号文 C_{AES}（対称鍵暗号）
4. $(C_{\mathbf{RSA}}, C_{\mathbf{AES}})$ **を送信**

ハイブリッド方式が主流な理由は以下のとおりである。

- RSA 暗号化は遅いため、大量データの暗号化には不向き
- 対称鍵暗号（AES 等）は高速だが、鍵を安全に共有する必要がある
- ハイブリッド方式では、RSA で小さな共通鍵だけを安全に送り、本文は AES で暗号化することで、速度と運用性を両立している

このように、実際のシステムにおける RSA の「暗号化」は、共通鍵を暗号化して送る**鍵輸送**として使われている。ただし、すべてのシステムが RSA 署名・RSA 鍵輸送の両方を採用しているわけではない。現状の使われ方を以下にまとめる。

[*1] 鍵カプセル化メカニズム（KEM）の一般的なガイドラインについては NIST SP 800-227 (2025) を参照。

システム	RSA 署名	RSA 鍵輸送	備考
HTTPS（TLS 1.3）	√ ※1	–	鍵交換は (EC)DHE に移行済み ※2。サーバ証明書は RSA/ECDSA が広く併用
メール暗号化（S/MIME、PGP）	√	√	RSA 鍵輸送が広く使われている。ECDH 等の他の鍵方式も並存
鍵管理システム（HSM、KMS）	√	△	鍵ラップのデフォルトは対称鍵（AES Key Wrap）。RSA による鍵輸送は BYOK/外部鍵インポート等の特定用途に限られる

√ = RSA が広く選択肢として残っている。△ = 限定用途で利用されるが主流ではない。– = 該当なし／使用されない。
※ 1: TLS 1.3 サーバ証明書として ECDSA も急速に普及している。
※ 2:「移行済み」は古典計算機下での安全性改善を指す。Diffie-Hellman 方式（楕円曲線版を含む）も離散対数問題に依存するため、Shor のアルゴリズムを想定すると影響を受ける。

2.2.2　RSA が破られる影響

RSA の秘密鍵が公開鍵から計算できるようになると、RSA を用いた署名と暗号化はいずれも根本から破綻する。ただし実務上の影響の現れ方は用途で異なる。

- **暗号化**
  - RSA が破られてからはもちろん危険だが、将来の復号を見越して、暗号文が今から収集され得る。
- **署名**
  - 偽造は RSA が破られた時点でのなりすまし・改ざんとして顕在化する。

2.2.3　署名が破られた場合の影響

署名が破られると、攻撃者は正規の署名を偽造できるようになる。これにより、以下のような攻撃が可能になる。

中間者攻撃（MITM）

ブラウザは CA（認証局）の署名を検証して証明書チェーンを信頼している。RSA が破られると、RSA 証明書を使うサーバの秘密鍵は公開鍵から導出可能になり、そのサーバになりすました TLS 接続が可能になる。さらに、信頼された CA が RSA 鍵で署名している場合には、その CA の秘密鍵も同様に導出可能となり、攻撃者は任意のドメイン向けの証明書を偽造できる。現実のパブリック PKI では大多数のルート CA が RSA 鍵を採用しているため、ECDSA 証明書を運用しているサーバでも、証明書チェーン上のいずれかに RSA があれば連鎖的に影響を受ける。

攻撃者が通信経路に介入できる状況（Wi-Fi、社内 LAN、DNS 汚染等）では以下の危険がある。

- ブラウザは警告を出さず、ユーザーは URL バーを含めて本物のサイトと区別できない
- 銀行、EC サイト、SNS 等への通信を傍受し、認証情報や決済情報を盗める

コード署名の偽造

コード署名は「特定の鍵の所有者が署名した」「配布物が改ざんされていない」ことを確認する仕組みであり、ソフトウェアの無害性を保証するものではない。しかし実務上、OS やセキュリティソフトは署名の有無を信頼判断の重要な要素として扱い、署名のないソフトウェアには警告を出している。RSA が破られると、公開鍵から秘密鍵が導出可能になり、正規ベンダのコード署名鍵や、コード署名 CA の署名鍵が偽造可能となる。結果として以下のような攻撃が可能になる。

- マルウェアに正規ベンダの署名を付けて配布でき、多くの環境で検知・警告のハードルを下げる
- 正規のソフトウェア更新に見せかけてバックドアを仕込むことで、サプライチェーン攻撃の成功確率や影響範囲が大きく増す

その他の影響

- **mTLS / クライアント証明書**
  - 社内システムや API 間通信では「相互 TLS（mTLS）」により、サーバだけでなくクライアント側も証明書で認証している
  - RSA が破られると
    - 攻撃者が正規の社員やシステムになりすまして社内ネットワークに侵入
    - 機密データの窃取、ランサムウェアの展開、内部システムの改ざん
- **VPN / ネットワーク機器認証**
  - 企業の VPN は証明書でゲートウェイやクライアントを認証し、不正なアクセスを防いでいる
  - RSA が破られると
    - 偽の VPN ゲートウェイに誘導
    - 攻撃者が正規ユーザーとして VPN 接続できる
    - リモートワーク環境からの社内ネットワーク侵入、通信の傍受
- **S/MIME 署名**

  - 企業間の重要なメール（契約、送金指示など）では「署名付きメール」で送信者の本人確認を行っている
  - RSA が破られると
    * CFO や法務部門になりすました署名付きメールを送信
    * S/MIME 署名を本人確認の根拠にしている組織では、署名検証が通るため、受信者・自動フィルタの双方が偽装を見抜きにくくなり、BEC の被害拡大につながる
- **文書署名・電子契約**
  - PDF 署名や電子契約サービスでは、署名によって「誰が承認したか」を証明し、改ざんを検知している
  - RSA が破られると
    * 契約書や承認文書を偽造できる
    * 不正な契約の成立、監査証跡の信頼性喪失、法的紛争

2.2.4 暗号化が破られた場合の影響

暗号化が破られると、攻撃者は RSA で暗号化されたデータを復号できるようになる。

過去のメールの復号（S/MIME、PGP）

- **S/MIME**
  - メールの暗号化・署名のための規格。通常のメールでは有効になっておらず、企業が証明書を取得して意図的に導入・設定する必要がある
- **PGP**
  - ファイルやメールの暗号化のためのソフトウェア。セキュリティを重視する個人や組織で使われている

これらで暗号化された企業間の機密メールが、RSA が破られると復号される。

- 過去に傍受・保存されていた暗号化メールが復号される
- M&A 交渉、契約内容、知的財産、医療情報などが漏洩

鍵管理システムの突破（HSM、KMS）

- **HSM（ハードウェアセキュリティモジュール）**
  - 暗号鍵を安全に保管・管理するための専用ハードウェア。銀行、政府機関、CA、決済システム、クラウド KMS 内部など高い秘匿性が要求される領域で広く利用されている
- **KMS（鍵管理サービス）**
  - クラウドベースの鍵管理サービス。AWS KMS、Azure Key Vault、Google Cloud KMS など

(※) KMS/HSM の実装によっては RSA を使わない（例えば AWS KMS 等の主要クラウド KMS のデフォルト鍵ラップは AES 対称鍵であり、RSA は外部鍵インポート/BYOK 等の特定用途に限られる）。

RSA で鍵ラップしている構成では、データベースやクラウドストレージの暗号化鍵がこれらのシステムで保護されている。RSA が破られると、以下のような影響がある。

- 暗号化鍵を保護している RSA 鍵が解読され、データ暗号化鍵が漏洩
- その範囲のデータベース・クラウドストレージに保存された機密データが復号可能に

コラム：量子コンピュータとブロックチェーンの関係

（※ 1.1.4 節で楕円曲線暗号は本書の範囲外としていますが、本コラムでは FAQ として短く触れます。）

「RSA が破られたら Bitcoin も危ないのか？」という疑問を持つ読者もいるかもしれません。Bitcoin/Ethereum は RSA を使っていないため、RSA **だけ**が破られる場合には直接の影響はありません。

チェーン	署名方式
Bitcoin	ECDSA（secp256k1）、Schnorr（Taproot 以降）
Ethereum	ECDSA（secp256k1）

しかし、本書で扱う Shor のアルゴリズムは素因数分解**だけ**を解くわけではありません。Shor は離散対数問題も多項式時間で解きます。ECDSA/Schnorr の安全性は楕円曲線上の離散対数問題に依存しているため、**Shor が実行可能な量子コンピュータが実現すれば、Bitcoin/Ethereum の署名方式も破られます**。つまり、攻撃者は公開鍵から秘密鍵を導出し、他人のコインを勝手に送金できてしまいます。

ただし、実際の攻撃可能性は、対象チェーンで公開鍵がいつ露出するかにも依存します。たとえば Bitcoin では、アドレス方式によっては公開鍵が支払い時まで露出しません。そのため量子攻撃が成立するには「公開鍵が既に公開されている未使用出力（UTXO）を狙う」か「支払いトランザクションがブロードキャストされてから承認されるまでの間に秘密鍵を導出する」といった条件が絡みます。

さらに、ブロックチェーンの周辺インフラ（取引所、ウォレットサービス、API 等）は TLS/PKI 基盤に依存しており、RSA 署名の証明書が使われている場合には中間者攻撃のリスクも生じます（取引所へのログイン情報の窃取、送金先アドレスの改ざん等）。

このように、量子コンピュータの脅威は RSA に限定されるものではなく、現在広く使われている公開鍵暗号全般に及びます。

2.3　RSA 暗号を説明するための前提条件

ここまで、RSA が破られた場合に署名の偽造や暗号文の復号がどのような被害を引き起こすかを見てきた。ここからは、RSA 暗号がなぜ安全とされているのか、その仕組みと手順について議論する。ただし、実運用の RSA 暗号をそのまま扱うと複雑になるため、本質に影響しない部分をシンプル化して説明する。

以下に、実運用との違いとその妥当性を整理する。

本章で用いる表記を以下に定義する。

表 2.3　本章でのシンプル化と実運用の比較

シンプル化	実運用	本章での扱い	妥当性
登場人物を 3 者に限定	多数の関係者（CA、中間者等）	Alice（送信者）、Bob（受信者）、Eve（盗聴者）	RSA の暗号化・復号・解読の本質は送信者・受信者・盗聴者の 3 者関係に集約される
鍵輸送部分のみ	RSA で共通鍵 K を暗号化（鍵輸送）し、本文は AES で暗号化	RSA による鍵輸送のみを扱う（AES 部分は省略）	Shor アルゴリズムが脅かすのは RSA 部分であり、AES 部分は独立している
パディングなし	OAEP/PSS を施してから暗号化・署名	パディングなしの教科書 RSA	実運用では OAEP/PSS は不可欠だが、Shor との関係上、本書では RSA の代数的骨格に集中する

- M: 平文メッセージ（守りたいデータ、$0 \leq M < n$）（注: 実運用の鍵輸送文脈では、この M は共通鍵 K に相当する。本章では一般化して M と書く。）
- C: Ciphertext（暗号文）
- n: 公開される合成数（$n = p \times q$）
- p, q: 大きな素数（秘密）
- e: 公開指数（公開鍵の一部）
- d: 秘密指数（秘密鍵）

以上の前提条件のもとで、RSA 暗号の全体の流れは以下のようになる。

- **ステップ① 鍵生成（Bob）**
  - 2 つの大きな素数 p, q を選び、$n = p \times q$ を計算。公開鍵 (e, n) を公開し、p, q は秘密にする
- **ステップ② 公開鍵の公開（Bob）**
  - (e, n) を公開する
- **ステップ③ 暗号化（Alice）**
  - Bob の公開鍵を使い、平文 M から暗号文 C を計算
- **ステップ④ 送信（Alice）**
  - Alice が Bob に暗号文 C を送信（Eve が盗聴する可能性あり）
- **ステップ⑤ 復号（Bob）**
  - 秘密の p, q から秘密鍵 d を求め、C から M を復元

Eve は C と公開鍵 (e, n) を入手できるが、p, q を知らないため d を求められず、M を復元できない。

2.4 節でこの手順に必要な数学的道具を準備し、2.5 節で各ステップの詳細を見ていく。

2.4　RSA 暗号の手順を理解するための数学

以下では、RSA 暗号の仕組みを理解するために必要な数学的概念を紹介する。

表 2.4　RSA に必要な数学的概念

概念	RSA での役割
オイラー関数・オイラーの定理	Bob の復号が成功する理由（数学的根拠）
拡張ユークリッドアルゴリズム	Bob が秘密鍵 d を効率的に計算する
二乗算法	Alice と Bob が暗号化・復号の冪乗を効率的に計算する

2.4.1　オイラー関数

RSA の復号では $C^d \equiv M \pmod{n}$ が成り立つ必要がある。この関係を担保しているのがオイラーの定理であり、その理解にはまずオイラー関数を知る必要がある。

定義

$$\phi(m) := \#\{i \in \mathbb{N} \mid 1 \leq i \leq m, \gcd(i, m) = 1\}$$

つまり、m と互いに素な（最大公約数が 1 である）、m 以下の正の整数の個数である。

p, q が互いに異なる素数の場合に成り立つ性質

p, q を互いに異なる素数とすると以下の関係が成り立つ。

$$\phi(p \times q) = (p-1)(q-1) \tag{2.1}$$

- 定義通りの計算の例
  $\phi(15) = \#\{1, 2, 4, 7, 8, 11, 13, 14\} = 8$
- p, q が素数の場合の例
  $\phi(15) = (3-1)(5-1) = 2 \times 4 = 8$

コラム：$\phi(pq) = (p-1)(q-1)$ の証明

$n = pq$ のうち、n と互いに素**でない**数（つまり p または q で割り切れる数）を数えて引きます。

- p の倍数：$p, 2p, \ldots, qp \to q$ 個
- q の倍数：$q, 2q, \ldots, pq \to p$ 個
- 重複（pq の倍数）：$pq \to 1$ 個

$$\phi(n) = pq - (p + q - 1) = pq - p - q + 1 = (p-1)(q-1)$$

オイラーの定理

$$\gcd(M, n) = 1 \text{ のとき、} \quad M^{\phi(n)} \equiv 1 \pmod{n} \tag{2.2}$$

M と n が互いに素であれば、M を $\phi(n)$ 乗すると n を法として 1 と合同になる。まさにこの関係は、Bob が RSA の復号が成功する数学的根拠となっている。

コラム：オイラーの定理の証明

n と互いに素な数を $a_1, a_2, ..., a_{\phi(n)}$ とします。M をこれらに掛けた $Ma_1, Ma_2, ..., Ma_{\phi(n)}$ も、$\bmod n$ で見ると同じ集合の並べ替えになります（M と n が互いに素なので、Ma_i も n と互いに素になります。また $Ma_i \equiv Ma_j \pmod{n}$ ならば $M(a_i - a_j) \equiv 0 \pmod{n}$ で、$\gcd(M, n) = 1$ なので $a_i \equiv a_j \pmod{n}$ となり、i=j でなければ異なります)。
したがって、両辺の積を取ると以下のように書けます。

$$Ma_1 \cdot Ma_2 \cdots Ma_{\phi(n)} \equiv a_1 \cdot a_2 \cdots a_{\phi(n)} \pmod{n}$$

左辺は以下のように整理することができます。

$$M^{\phi(n)} \cdot (a_1 \cdot a_2 \cdots a_{\phi(n)}) \equiv a_1 \cdot a_2 \cdots a_{\phi(n)} \pmod{n}$$

$a_1 \cdot a_2 \cdots a_{\phi(n)}$ は n と互いに素なので $\bmod n$ での逆元が存在します。両辺にその逆元を掛けると以下が得られます。

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

2.4.2　拡張ユークリッドアルゴリズム（Extended Euclidean Algorithm）

RSA では、$d \cdot e \equiv 1 \pmod{\phi(n)}$ を満たす d を求める必要がある。この問題は「$d \cdot e + t \cdot \phi(n) = 1$ を満たす整数 d, t を求める」と言い換えられ、**拡張ユークリッドアルゴリズム**で効率的に解ける。

アルゴリズムの手順

$\gcd(a, b) = 1$ のとき、$ax + by = 1$ を満たす整数 x, y を求める

1. **ユークリッドの互除法**で $\gcd(a, b)$ を計算しながら、商と余りを記録
2. **逆算**で x, y を求める

具体例：$e = 7,\ \phi(n) = 40$ の場合

$7d + 40t = 1$ を満たす d を求める。

Step 1: 互除法（$\gcd(40, 7)$ を計算）

$$
\begin{aligned}
40 &= 7 \times 5 + 5 && \cdots\text{(i)} \\
7 &= 5 \times 1 + 2 && \cdots\text{(ii)} \\
5 &= 2 \times 2 + 1 && \cdots\text{(iii)} \quad \leftarrow \gcd = 1 \text{ に到達} \\
2 &= 1 \times 2 + 0 && \cdots\text{(iv)}
\end{aligned}
$$

Step 2: 逆算（$1 = 7x + 40y$ の形に変形）

$$
\begin{aligned}
\text{(iii) より：}\quad & 1 = 5 - 2 \times 2 \\
\text{(ii) を代入：}\quad & 1 = 5 - (7 - 5 \times 1) \times 2 = 5 \times 3 - 7 \times 2 \\
\text{(i) を代入：}\quad & 1 = (40 - 7 \times 5) \times 3 - 7 \times 2 = 40 \times 3 - 7 \times 17
\end{aligned}
$$

よって $1 = 40 \times 3 + 7 \times (-17)$、つまり $7 \times (-17) \equiv 1 \pmod{40}$

$d = -17 \equiv 23 \pmod{40}$ が答え。

検証

$7 \times 23 = 161 = 40 \times 4 + 1 \equiv 1 \pmod{40}$

計算量

- 互除法は最悪でも $O(L)$ 回の反復で終了する（$L = \log_2 n$ は入力のビット長。最悪ケースは連続するフィボナッチ数で達成される）
- 各反復の除算・剰余計算コストは多倍長演算に依存するが、全体として**多項式時間**で実行できる（反復回数は $O(L)$、各反復の多倍長除算は素朴に $O(L^2)$ なので、合計で素朴には $O(L^3)$ 程度の上界が得られる）

コラム：計算量の漸近記法（O）

本節以降、計算量を評価するために O 記法を使用します。

O 記法（上界）

- $f(n) = O(g(n))$ は、「f の増加速度は g を超えない」という意味です
- 例：$3n^2 + 5n = O(n^2)$
  - n が大きくなると、$3n^2$ の項が支配的になります
  - 定数倍（3）や低次項（$5n$）は無視できます
  - 結果として n^2 に比例する程度で増加します

2.4.3 二乗算法（Binary Exponentiation / 繰り返し二乗法）

RSA の暗号化 $C = M^e \bmod n$ や復号 $M = C^d \bmod n$ では、巨大な指数の冪乗を計算する必要がある。単純に M を e 回掛けると $O(e)$ 回の乗算が必要だが、**二乗算法**を使えば $O(\log e)$ 回で済む。

アルゴリズムの手順

指数 e を二進数で表し、各ビットに対応する二乗を組み合わせる。

$$M^e = M^{\sum_i e_i \cdot 2^i} = \prod_{i:e_i=1} M^{2^i}$$

1. e を二進数に変換
2. $M, M^2, M^4, M^8, \ldots$ を順次計算（前の結果を二乗）
3. e のビットが 1 の位置に対応する項を掛け合わせる

具体例：$7^{13} \bmod 15$ を計算

（注意: この例は二乗算法の動作を示すための独立した例であり、上の EEA 例 $(e = 7, \phi(n) = 40)$ とは無関係である。）

Step 1: 13 を二進数に変換 → $13 = 1101_2 = 8 + 4 + 1$

つまり $7^{13} = 7^8 \times 7^4 \times 7^1$

Step 2: 二乗を順次計算（mod15）

$$\begin{aligned}
7^1 &= 7 \\
7^2 &= 49 \equiv 4 \pmod{15} \\
7^4 &= 4^2 = 16 \equiv 1 \pmod{15} \\
7^8 &= 1^2 = 1 \pmod{15}
\end{aligned}$$

Step 3: 必要な項を掛け合わせる

$$7^{13} = 7^8 \times 7^4 \times 7^1 = 1 \times 1 \times 7 = 7 \pmod{15}$$

検証: 直接計算すると $7^{13} = 96889010407 \to 96889010407 \bmod 15 = 7$ ✓

計算量

- 指数のビット長は最大 $O(L)$（$L = \log_2 n$、秘密指数 d の場合）。公開指数 $e = 65537$ の場合は 17 ビット固定なので $O(1)$ で済み、暗号化は特に高速
- 必要な乗算回数は $O(\log e)$ 回（最大 $O(L)$ 回）
- 各乗算は L ビット同士の多倍長乗算（素朴には $O(L^2)$）
- **全体**: $O(L) \times O(L^2) = O(L^3)$（多項式時間）

2.5　RSA 暗号の手順

以下では RSA 暗号で、送信者 Alice から正当な受信者 Bob に秘密のメッセージを送る手順をたどり、仕組みを理解する。また、Alice から Bob への暗号化されたメッセージを盗聴者 Eve に盗聴されても安全と考える根拠を示す。

2.5.1　ステップ① メッセージ受信者側の事前準備（Bob が実行）

1. **2 つの大きな異なる素数 p, q を選ぶ**（近すぎないこと）
   - 実用では 2048 bits（各素数が約 1024 bits）
2. **合成数 n を計算**
$$n = p \times q \tag{2.3}$$
3. **公開指数 e を選ぶ**
   - **条件**
     $\gcd(e, \phi(n)) = 1$（e と $\phi(n)$ が互いに素）
   - **一般的な選択**
     $e = 2^{16} + 1 = 65537$（この値は素数であり、計算効率も良いためよく使われている）

2.5.2　ステップ② メッセージ受信者側の公開鍵の公開（Bob が実行）

Bob は (e, n) を公開する。

- **公開される情報**
  $(e, n = p \times q)$
- **秘密にする情報**
  (p, q)

2.5.3　ステップ③ メッセージ送信者側の暗号化（Alice が実行）

Alice は、平文 M を以下のように暗号化する。

$$C \equiv M^e \pmod{n} \tag{2.4}$$

コラム：教科書 RSA と実運用の違い（パディング）

上記の式 (2.4) は「教科書 RSA」と呼ばれる数学的な原理です。実際のシステムでは、平文 m をそのまま暗号化するのではなく、**パディング**と呼ばれる前処理を施してから暗号化します。

	式
教科書 RSA	$M = m,\ C \equiv M^e \pmod{n}$
実運用（OAEP）	$M := \mathrm{OAEPEncode}(m; r),\ C \equiv M^e \pmod{n}$

ここで r は暗号化のたびに Alice が生成するランダム値です。OAEPEncode はハッシュ関数と XOR 演算で m と r を一つのブロックに混ぜ合わせる関数で、RSAES-OAEP として標準化されています。r はブロック内に埋め込まれて RSA 暗号化されるため、別途送信する必要はなく、外部に公開もされません。

この前処理により、同じ m を暗号化しても毎回異なる M（したがって異なる暗号文 C）が生成されます。教科書 RSA では同じ m に対して常に同じ C になる（決定的暗号）ため、攻撃者が暗号文を比較するだけで内容の一致を検知できてしまいます。また、教科書 RSA は乗法的準同型性（$E(M_1) \cdot E(M_2) \equiv E(M_1 \cdot M_2) \pmod{n}$）を持つため、暗号文を操作するだけで平文の関係を利用した攻撃（選択暗号文攻撃）が可能になります。

パディングが追加されても、Bob の復号は容易です。$C^d \pmod{n}$ を計算すると M が復元されます。$M = \mathrm{OAEPEncode}(m; r)$ なので、OAEPDecode により平文 m を取り出します。OAEPEncode は可逆に設計されているため、復号側に追加される計算は軽微です。

署名の場合も同様に、署名対象のハッシュ値にランダム値を付加してから署名する方式（RSASSA-PSS）が標準です。

教科書 RSA をそのまま実運用に使うべきではなく、新規設計では RSAES-OAEP ／ RSASSA-PSS の採用が標準的です（RFC 8017 はこれらを含む RSA の各方式を仕様化しています）。本書では Shor アルゴリズムとの関係上、RSA の代数的骨格（$M^e \bmod n$ の構造）に集中するため、パディングの詳細は省略します。Shor アルゴリズムが攻撃するのはこの代数的骨格そのものであり、OAEP/PSS の有無にかかわらず脅威となります。

2.5.4　ステップ④ メッセージの送信

Alice は Bob に向けて暗号文 C を送信。この暗号文 C は盗聴者 Eve に盗聴される可能性がある。

2.5.5　ステップ⑤ メッセージ受信側の暗号文の復号（Bob が実行）

この時点で Bob が持っている情報

- p, q → ステップ①で選んだ 2 つの素数（Eve は知らない）
- $n = p \times q$ → 式 (2.3) で計算済み
- $e = 65537$ → ステップ①で選んだ公開指数

秘密指数 d の計算

Bob は、以下の式を満たす秘密指数 d（秘密鍵の主要成分）を用意する。実運用では d は鍵生成時に一度計算し、秘密鍵として保持する。

$$d \cdot e \equiv 1 \pmod{\phi(n)} \tag{2.5}$$

Bob は p, q を知っているため、$\phi(n)$ は、$\phi(n) = (p-1)(q-1)$ から簡単に計算可能である。

> **コラム：$\phi(n)$ と $\lambda(n)$ の関係**
> 本章では説明の簡略化のため $\phi(n)$ を用います。RSA の正当性条件はしばしば $\lambda(n) = \mathrm{lcm}(p-1, q-1)$ を用いて $ed \equiv 1 \pmod{\lambda(n)}$ と表されます（例：RFC 8017）。一方で、実装・規格により d を $e^{-1} \bmod \phi(n)$ として求める場合もあります。なお $\lambda(n) \mid \phi(n)$ なので、$ed \equiv 1 \pmod{\phi(n)}$ を満たせば $ed \equiv 1 \pmod{\lambda(n)}$ も満たします。

式 (2.5) について、$d \cdot e \equiv 1 \pmod{\phi(n)}$ とは、$d \cdot e$ を $\phi(n)$ で割った余りが 1 になるということである。これは、ある整数 t が存在して以下が成り立つことと同値である。

$$d \cdot e + t \cdot \phi(n) = 1$$

e と $\phi(n)$ は既知なので、この式は d と t を未知数とする一次不定方程式である。これは 2.4.2 節で説明した拡張ユークリッドアルゴリズムそのものであり、多項式時間で計算できる。

なお、復号の証明では $ed - 1$ が $\phi(n)$ の正の整数倍であることを利用する。記号の混同を避けるため、$ed = 1 + k\phi(n)$（$k \geq 1$）と書く。

復号の実行

Bob は、用意した秘密指数 d を使って以下のようにメッセージを復号する。

$$M \equiv C^d \pmod{n} \tag{2.6}$$

$$C^d \equiv (M^e)^d \equiv M^{ed} \equiv M^{1+k\phi(n)} \equiv M \cdot (M^{\phi(n)})^k \equiv M \pmod{n} \tag{2.7}$$

この証明は $\gcd(M, n) = 1$ の前提のもとで、式 (2.2) のオイラーの定理 $M^{\phi(n)} \equiv 1 \pmod{n}$ を用いている。

$\gcd(M, n) \neq 1$ となる場合（M が p または q の倍数になる場合）には、上の証明はそのままでは使えない。この場合は中国の剰余定理（CRT）を用いて $\bmod p$ と $\bmod q$ で個別に $M^{ed} \equiv M$ を示すことで、$\gcd(M, n)$ によらず全 $M \in \{0, 1, \ldots, n-1\}$ に対して $M^{ed} \equiv M \pmod{n}$ が成り立つことが示せる（暗号学的にはこちらが標準の証明）。具体的には、$\bmod p$ について場合分けする。

- $\gcd(M, p) = 1$ のとき：フェルマーの小定理より $M^{p-1} \equiv 1 \pmod{p}$。$(p-1) \mid \phi(n)$ なので $\phi(n) = (p-1)q'$ と書けて、$ed = 1 + k\phi(n)$ より $M^{ed} = M \cdot M^{k\phi(n)} = M \cdot (M^{p-1})^{kq'} \equiv M \pmod{p}$。
- $p \mid M$ のとき：$M \equiv 0 \pmod{p}$ なので、$M^{ed} \equiv 0 \equiv M \pmod{p}$ が自明に成立する。

$\bmod q$ についても同様に成立する。CRT で結合すると、全 M に対して $M^{ed} \equiv M \pmod{n}$ が成り立つ。

この計算は**二乗算法**により多項式時間で実行可能である。

2.6 RSA の安全性 — 計算量の視点

以後、計算量は入力サイズ $L = \log_2 n$（n のビット長）に対して評価する。例えば $O(n) = O(2^L)$ であり、入力サイズに対して指数時間となる。

式 (2.5) の d が求まれば、式 (2.7) のように簡単に誰でも復号が可能である。d を求めるためのパラメータとして、e は公開しているため、**正当な受信者 Bob と盗聴者 Eve の本質的な違いは「$\phi(n)$ を求める難しさ」である。**

2.6.1 正当側の操作はすべて多項式時間

表 2.5 に示す通り、正当側（Alice および Bob）の操作はすべて**多項式時間**で完了する。

表 2.5 正当側の操作と計算量

#	主体	RSA ステップ	操作	計算量
1	Bob	① 鍵生成	素数生成（Miller-Rabin 判定（確率的素数判定法。t ラウンド実施で誤判定確率を 4^{-t} 以下に抑えられる）、期待回数 $O(L)$）	$O(tL^3)$（1 候補あたり、t は Miller-Rabin のラウンド数）。期待総計 $O(tL^4)$
2	Bob	① 鍵生成	$n = p \times q$ の乗算	$O(L^2)$
3	Bob	① 鍵生成	公開指数 e の選択（$e = 65537$ 等）	$O(1)$
4	Bob	① 鍵生成	$\phi(n) = (p-1)(q-1)$ の計算	$O(L^2)$
5	Bob	① 鍵生成	秘密指数 d の計算（拡張 Euclid アルゴリズム）	$O(L^3)$
6	Alice	③ 暗号化	$C = M^e \bmod n$（二乗算法）	$O(L^3)$
7	Bob	⑤ 復号	$M = C^d \bmod n$（二乗算法）	$O(L^3)$

(※) 実装では通常 CRT（p, q それぞれで冪乗して合成）で復号・署名生成を高速化することになるが、計算量クラスとしては依然多項式時間である。

(※※) 期待回数 $O(L)$ は素数定理に由来する：ランダムに L ビットの奇数を引くと素数である確率は約 $1/L$ なので、期待 $O(L)$ 個の候補を試すことになる。各候補について Miller-Rabin を t ラウンド実行するので、全体で $O(tL^4)$ ビット演算となる。

2.6.2　なぜ Eve は解読できないのか

一方で、Eve は n は知っているが p, q を知らないため、$\phi(n)$ を直接計算することができない。$\phi(n)$ を求める最も自然な経路は n を素因数分解することであり（素因数分解を経由しない攻撃経路については 2.6.3 節で後述する）、現在知られている最良の古典アルゴリズム（GNFS）の計算量は以下のように表される。

$$L_n[1/3, c] = \exp\left(c \cdot (\ln n)^{1/3} (\ln \ln n)^{2/3}\right)$$

ここで n は素因数分解したい数、c は定数（GNFS では $(64/9)^{1/3} \approx 1.923$）である。重要なのは $\exp(\cdots)$ の構造で、n が大きくなると指数部が増大し、計算量が爆発的に膨らむ。これは**準指数時間**と呼ばれ、Bob の多項式時間とは桁違いに遅い。この差こそが、RSA 暗号の安全性の数学的根拠である。素因数分解が多項式時間で解けないことは証明されていないが、広く困難だと信じられている。

実際に、GNFS による素因数分解の実績を見ると、現在標準の 2048 ビットは人類には未解読である。

n のビット数	具体例	備考
512 ビット	RSA-155（155 桁）	1999 年に解読成功
768 ビット	RSA-768（232 桁）	2009 年に解読成功
795 ビット	RSA-240（240 桁）	2019 年に解読成功
829 ビット	RSA-250（250 桁）	2020 年に解読成功 ※
1024 ビット	RSA-1024	未解読（非推奨）
2048 ビット	RSA-2048	現在の標準（NIST SP 800-57: 112 ビット安全強度。具体的評価は 3.5.6 節参照）
3072 ビット	RSA-3072	128 ビット安全強度
4096 ビット	RSA-4096	性能とのトレードオフ

※ 2026 年 4 月時点での一般目的アルゴリズムによる公開最大記録。RSA Factoring Challenge 自体は 2007 年に終了しており、上記の解読はいずれも学術的な「記録計算」である。

2.6.3　素因数分解を経由しない攻撃（RSA 問題）

ここまでは「秘密鍵 d を復元して復号する」という最も標準的な攻撃経路を示した。理論的には d や $\phi(n)$ を求めずに $C \equiv M^e \pmod{n}$ の逆像として M を直接求める攻撃（RSA 問題）も考えられるが、これが素因数分解と同程度に難しいかどうかは一般には未解決である。

2.7　まとめ

本章では、RSA 暗号の仕組みを解説した。RSA 暗号の安全性は、以下の計算量的困難さに依存している。

1. **$n = p \times q$ の素因数分解は、現在知られている最良の古典アルゴリズム（GNFS）では準指数時間かかる**
2. **正当な受信者（Bob）は p, q を知っているため多項式時間で復号できる**
3. **盗聴者（Eve）にとっては、素因数分解以外に実用的な攻撃経路は知られておらず、現実的な計算資源では困難と考えられている**（RSA 問題と素因数分解の等価性は未証明。詳細は 2.6.3 節）

以上の計算量的安全性を根拠に、RSA 暗号は 40 年以上利用されている。

しかし、もし仮に素因数分解が簡単に解けるようになった場合、RSA 暗号の安全性は破られてしまう。そして量子コンピュータを利用した Shor のアルゴリズムでは、素因数分解が多項式時間で解けることが示されている。

次章では、この Shor のアルゴリズムについて詳しく解説する。

第 3 章

Shor のアルゴリズム

3.1 はじめに

前章では、RSA 暗号の安全性は素因数分解の計算量的安全性に帰着することを述べた。一方、量子計算では、理論的に素因数分解の計算量的安全性を破るアルゴリズムが発見されている。

3.1.1 本章の前提と範囲

量子コンピュータとは、量子力学の法則に基づいて計算を行う装置である。計算の流れは大きく 3 ステップからなる。

1. **初期状態の準備**
   量子ビットを所定の状態にセットする
2. **量子ゲートの作用**
   ユニタリ演算子を順に作用させ、状態を変換する
3. **測定＋古典計算**
   量子ビットを測定し、その結果をもとに古典計算で欲しい答えを求める

以下では、この過程で使う表記を整理する。

- $|0\rangle, |1\rangle$：量子ビットの状態（ケット表記）。量子コンピュータでは物理的に区別可能な 2 状態を選び、これらを $|0\rangle, |1\rangle$ と表す。「区別可能」を数学的に表現すると「直交している」ということであり、内積は $\langle 0|1\rangle = 0$、$\langle 0|0\rangle = \langle 1|1\rangle = 1$ となる
- 量子ビットは $|0\rangle$ と $|1\rangle$ の**重ね合わせ**状態 $\alpha|0\rangle+\beta|1\rangle$ をとれる。α, β は複素数で、$|\alpha|^2+|\beta|^2 = 1$ を満たす
- $\langle\psi|$：ブラ表記。ケット $|\psi\rangle$ に対応する行ベクトルで、内積 $\langle\psi|\phi\rangle$ を計算するときに使う
- $\otimes$：テンソル積。$|0\rangle \otimes |1\rangle$ は 2 つの量子ビットの合成を表す
- U：量子ゲート（ユニタリ演算子）。入力状態を別の状態に変換する
- $e^{i\theta}$：複素指数。複素平面上の単位円上の点を表す（$|e^{i\theta}| = 1$）。本章ではユニタリ演算子の固有値が $e^{2\pi i\varphi}$ の形で現れ、この φ を読み取ることが目標となる

本章で扱う**測定**は、計算基底（$|0\rangle, |1\rangle$）での測定のみである。測定すると確率的にどちらかの状態が得られ、その確率はボルンの規則 $P(z) = |\langle z|\psi\rangle|^2$ で決まる。

本章では、これらのルールを受け入れ、そこから先の計算を追う。固有値・複素数・線形代数などの数学的基礎は、必要に応じて読者自身で補っていただきたい。

3.1.2　本章の話の流れ

Shor のアルゴリズム [1] を理解するために、本章では以下の順序で議論を進める。

位数発見問題

Shor のアルゴリズムは「素因数分解」を直接解くのではなく、「**位数発見**」という別の問題に帰着させる。実は、n を法とした a の位数さえ分かれば、素因数は古典計算で簡単に求まる。この帰着は純粋に数論の話であり、量子計算とは無関係である。

量子位相推定（QPE）

では、a の位数をどう求めるか？　古典計算では位数を効率的に求める方法は知られていない。ここで量子計算の出番となる。ここでは直接 Shor のアルゴリズムの話に進む前に、**量子位相推定（QPE）**について述べる。QPE は、ユニタリ演算子の固有値（位相）を読み取る汎用的な量子アルゴリズムである。Shor のアルゴリズムは、この QPE を「位数発見」に特化させた形と見なせる。

Shor のアルゴリズムの手順

前二節を踏まえ、理想的な量子コンピュータを前提に、Shor のアルゴリズム全体の手順を具体的に示す。回路構成、測定、古典後処理の流れを追う。最後に手順のまとめを示す。

実用性の分析

まず、理論と実装のギャップを説明する（なぜ a^y を a^{2^k} で実装するのか）。次に、成功確率を評価し、それを基にアルゴリズムの計算量を古典アルゴリズムと比較する。なぜ「準指数時間→多項式時間」という飛躍が起きるのかを定量的に理解する。最後に、現状の技術では Shor のアルゴリズムが実現できない理由を議論する。

本書を読めば、Shor アルゴリズムの各ステップで何をしているかを理解し、古典アルゴリズムと比較したとき、どういう条件で、どの程度速いのかのイメージをつかみ、現実の技術とのギャップを正しく認識できるようになることを目指す。

3.2　位数発見問題（Order Finding Problem）

Shor のアルゴリズムは「素因数分解」を直接解くのではなく、「**位数発見**」という別の問題に帰着させる。本章では、「**n を法とした a の位数 r が分かれば、$n = pq$ の素因数が高確率で求められる**」ことを述べる。

用語の整理 RSA 暗号では、2 つの異なる大きな素数 p, q の積 $n = pq$ を使う。このような数を**半素数（semiprime）**と呼ぶ。本章では、この n を因数分解する問題を扱う。

3.2.1　位数の定義

a と n を互いに素な自然数（$\gcd(a, n) = 1$）とする。

$$a^r \equiv 1 \pmod{n} \tag{3.1}$$

を満たす**最小の正の整数** r を、n を法とした a の **位数 (order)** と呼び、以降は「$\mathrm{ord}_n(a) = r_{(n,a)}$」と表記する。

具体例：$n = 15$, $a = 7$ **の時の位数** $\mathsf{ord}_{15}(7)$

7 の累乗を mod15 で計算し、1 になるときの指数を見つければ良い。

- $7^1 \bmod 15 = 7$
- $7^2 \bmod 15 = 49 \bmod 15 = 4$
- $7^3 \bmod 15 = 7 \cdot 4 = 28 \bmod 15 = 13$
- $7^4 \bmod 15 = 7 \cdot 13 = 91 \bmod 15 = 1$

と計算できるので、$\mathrm{ord}_{15}(7) = 4$ であり、15 を法とした 7 の位数は 4 である。

具体例：$n = 15$, $a = 11$ **の時の位数** $\mathsf{ord}_{15}(11)$

同様に、11 の累乗を mod15 で計算する。

- $11^1 \bmod 15 = 11$
- $11^2 \bmod 15 = 121 \bmod 15 = 1$

よって $\mathrm{ord}_{15}(11) = 2$ であり、15 を法とした 11 の位数は 2 である。

ここでポイントとなるのは、同じ n でも $\mathrm{ord}_n(a)$ は a に依存して変わることである。上記の例では、$\mathrm{ord}_{15}(7) = 4$、$\mathrm{ord}_{15}(11) = 2$ となっている。

Note: 位数から素因数を求める過程での確率的失敗

位数から素因数を求める過程では、確率的に失敗することがあります（r が奇数、または $a^{r/2} \equiv -1 \pmod{n}$）。これは乱択アルゴリズムとしての性質であり、量子力学の測定が確率的であることとは無関係です。

失敗したときは、a を変えて別の位数を試せばよいです。やり直しにかかる計算量は微々たるものです（詳細は 3.5.4 節のコラム「a の選び方と成功確率」を参照）。

※ 量子測定に由来する失敗（QPE で $s = 0$ が測定される場合など）については、3.3 節・3.4 節で別途議論します。

3.2.2　位数から素因数分解へ

冒頭でも述べた通り、n を法とした a の位数 r が求まれば、以下の流れで高確率で n の素因数 p, q を求めることができる。

ステップ 1 : r が偶数であることを確認 (奇数なら失敗)

式 (3.1) $a^r \equiv 1 \pmod{n}$ と r が偶数であることから、以下の式が成り立つ

$$a^r - 1 = kn \tag{3.2}$$

$$(a^{r/2}+1)(a^{r/2}-1) = kn \tag{3.3}$$

ここで、k は整数である。式 (3.3) では、$r/2$ は正の整数で、$1 \leq r/2 < r$ である。位数の最小性（式 (3.1) を満たす最小の正整数が r）より、$a^{r/2} \equiv 1 \pmod{n}$ は成り立たない。したがって、$(a^{r/2}-1)$ は n の倍数ではない。一方で、$(a^{r/2}+1)$ に関しては、n の倍数かを断定することはできない。

ステップ 2 :$(a^{r/2}+1)$ が n の倍数でないことを確認 (n の倍数なら失敗)

上記の確認は、繰り返し二乗法とユークリッド互除法の古典計算で簡単に確認することができる。これら 2 つの条件（r が偶数、かつ $(a^{r/2}+1), (a^{r/2}-1)$ のいずれも n の倍数でない）が満たされるとき、式 (3.3) より $(a^{r/2}+1)(a^{r/2}-1) = kn$ であり、$n = p \times q$ である。ここで p, q は素数なので、Euclid の補題により、それぞれ $(a^{r/2}+1)$ か $(a^{r/2}-1)$ のいずれかを割る。仮に p と q が同じ因子を割るとすると、$n = pq$ がその因子を割ることになり、ステップ 2 の条件（両因子が n の倍数でない）に矛盾する。よって p, q は $(a^{r/2}+1)$ と $(a^{r/2}-1)$ の片方ずつに分配される。したがって、gcd を取ることで非自明な因数を取り出せる。

$$p = \gcd(a^{r/2}+1, n), \quad q = \gcd(a^{r/2}-1, n) \tag{3.4}$$

（gcd の結果のどちらが p でどちらが q になるかは a に依存するが、$\{p, q\}$ 集合としては正しく取り出せる。）

ユークリッド互除法で多項式時間で求められる。

※ 素数の積 $n = p \times q$ に関して、n と互いに素な数 $a_1, a_2, a_3, \ldots$ を考えれば、それぞれ n を法とした位数 $r_{(n,a_1)}, r_{(n,a_2)}, \ldots$ が存在する。各ステップで失敗しても別の a を選んで位数を求め直せば良い。

以上より、n を法とした a の位数 r さえ求めることができれば、その後の処理は mod$n$ の計算と gcd 計算（ユークリッド互除法）だけで済み、古典計算で容易に素因数を得られることが分かった。

しかし、従来の古典的な計算モデルでは、一般の大きな合成数 n に対して a の位数 r を多項式時間で求める方法は知られていない（実質的に因数分解と同程度に難しいと考えられている）。

一方で量子計算では、**位数を多項式時間で求める手順（位相推定＋量子フーリエ変換）**が与えられており、これこそがまさに「Shor のアルゴリズム」である。

Shor のアルゴリズムは、RSA 暗号のような我々の世の中で非常に重要なトピックに対して、量子コンピュータが直接アプローチできる稀な例である。量子コンピュータが注目を集める最も大きな理由といっても過言ではない。

3.3　量子位相推定

Shor のアルゴリズムの話題に移る前に、量子位相推定について述べたい。量子位相推定は、ユニタリー演算子の固有値（の位相）を測定で読み取るアルゴリズムである。歴史的には Shor のアルゴリズムの方が先に考案されたが、体系的には、『**Shor のアルゴリズムは位相推定 (QPE) アルゴリズムの特別な場合**』という位置づけであり、Shor のアルゴリズムの手順を俯瞰的に理解するためには避けて通れないポイントである。

本章では、『**位相推定アルゴリズムの流れと理論**』を述べる。

3.3.1　量子位相推定の問題設定

ユニタリー演算子とは $U^\dagger U = I$（逆行列が複素共役転置に等しい）を満たす演算子であり、固有値の絶対値は必ず 1 になる。したがって、任意のユニタリー演算子 U とその固有状態 $|\psi\rangle$ に対して、固有値は以下のように書ける。

$$U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle \tag{3.5}$$

- U：ユニタリー演算子
- $|\psi\rangle$：U の固有状態（既知）
- $e^{2\pi i\varphi}$：固有値。$\varphi \in [0, 1)$ を**位相**と呼ぶ（未知）

問題設定としては、固有状態 $|\psi\rangle$ を持っているとき、位相 φ を測定で読み取りたい

3.3.2　量子位相推定のアルゴリズム

以下では、位相 φ を測定で読み取る手順を示す。

ステップ 1：初期状態を準備

量子位相推定では、以下の 2 種類の量子ビットを用意する。

① M 個の制御ビット（位相情報を読み取るためのビット列）

位相 φ を測定値として読み取るために、M 個の量子ビット $|0\rangle_{\mathcal{M}} = |0\rangle_1 \otimes |0\rangle_2 \otimes ... \otimes |0\rangle_M$ を用意する。この M は測定精度を決めるパラメータである。計算していけばわかるが、M ビットに設定したとき、位相 φ を 2^M 分割した精度で読み取ることができる。

② N 量子ビットのターゲットレジスタ（位相を読み取りたいユニタリ演算子 U の固有状態）

ユニタリー演算子 U の固有状態 $|\psi\rangle_{\mathcal{N}}$ を用意する。

U が d 次元空間に作用する場合、ターゲットレジスタは N 量子ビットを取り、$2^N \geq d$ を満たすようにする（実際には、補助量子ビットを使うなどの理由で増やすこともある）。

以上をまとめて、初期状態を以下のように記載する。

$$|\psi_0\rangle = |0\rangle_{\mathcal{M}} \otimes |\psi\rangle_{\mathcal{N}} \tag{3.6}$$

コラム：計算基底と正規直交性

N ビットの量子レジスタの計算基底 $|m\rangle$（$m = 0, 1, \ldots, 2^N - 1$）は、整数 m の二進表現に対応します。例えば $N = 4$ の場合以下のように書けます。

- $|0\rangle = |0\rangle \otimes |0\rangle \otimes |0\rangle \otimes |0\rangle$
- $|1\rangle = |0\rangle \otimes |0\rangle \otimes |0\rangle \otimes |1\rangle$
- $|7\rangle = |0\rangle \otimes |1\rangle \otimes |1\rangle \otimes |1\rangle$

異なる整数に対応する計算基底は、少なくとも 1 ビットが異なるため直交します。したがって、計算基底は正規直交基底をなすことになります。

$$\langle m'|m\rangle = \delta_{m'm} = \begin{cases} 1 & (m = m') \\ 0 & (m \neq m') \end{cases}$$

コラム：計算基底での測定とは

「制御ビットを測定する」とは、各量子ビットを**計算基底**（$|0\rangle$ と $|1\rangle$）で測定することを意味します。

測定すると、各量子ビットは確率的に $|0\rangle$ または $|1\rangle$ のどちらかに収縮します。M 個の量子ビットを測定すると M ビットの二進数が得られ、これを整数として解釈したものが $z \in \{0, 1, \ldots, 2^M - 1\}$ です。

測定確率は**ボルンの規則**で決まります：$P(z) = |\langle z|\psi\rangle|^2$

物理的な実装は量子ビットの種類によって異なります。超伝導量子ビットでは読み出し共振器にマイクロ波を送り周波数シフトを検出します。イオントラップではレーザーを照射し、蛍光の有無で $|0\rangle$ と $|1\rangle$ を判定します。

ステップ 2：制御ビットにアダマールゲートを適用

初期状態にアダマールゲートを作用させることで、位相推定の並列計算の土台を準備する。

アダマールゲート U_H の定義は以下の通りである。

$$U_H : |0\rangle \longmapsto \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$
$$U_H : |1\rangle \longmapsto \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

コラム：量子ゲートとユニタリ性

量子回路を構成するゲート（測定を除く）は、ユニタリ演算子でなければなりません。これは確率の総和が 1 であり続けること（ノルム保存）を保証するための要件であり、ユニタリでない操作は量子ゲートとして物理的に実装できません。本章で QFT や $U_{a,n}$ のユニタリ性を逐一確認するのは、それらが「紙の上の数式」ではなく実際に量子回路として実装可能であることを担保するためです。

アダマールゲートのユニタリ性の確認

アダマールゲートの行列表現は以下の通りです。

$$U_H = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$$

U_H は実行列かつ対称なので $U_H^\dagger = U_H$ であり、

$$U_H^2 = \frac{1}{2}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}2 & 0\\ 0 & 2\end{pmatrix} = I$$

したがって $U_H^\dagger U_H = U_H^2 = I$ が成り立ちます。同様に $U_H U_H^\dagger = U_H^2 = I$ も成り立つので、U_H はユニタリ演算子です。超伝導量子ビットでは特定の周波数・振幅のマイクロ波パルスとして実現されます。

本ステップでは、初期状態 $|\psi_0\rangle = |0\rangle_{\mathcal{M}} \otimes |\psi\rangle_{\mathcal{N}}$ の制御ビット（左側）にのみアダマールゲートを作用させ、ターゲットビット（右側）には恒等演算子 I（何もしない）を作用させる。

$$\left(\underbrace{U_H \otimes U_H \otimes \cdots \otimes U_H}_{M\text{個}}\right) \otimes I : |0\rangle_{\mathcal{M}} \otimes |\psi\rangle_{\mathcal{N}}$$

制御ビット部分 (左側) に注目すると、以下のように変換されることになる。

$$\underbrace{U_H \otimes U_H \otimes \cdots \otimes U_H}_{M\text{個}} |0\rangle_1 \otimes |0\rangle_2 \otimes \cdots \otimes |0\rangle_M$$

$$= U_H |0\rangle_1 \otimes U_H |0\rangle_2 \otimes \cdots \otimes U_H |0\rangle_M$$

$$= \frac{1}{\sqrt{2}}(|0\rangle_1 + |1\rangle_1) \otimes \frac{1}{\sqrt{2}}(|0\rangle_2 + |1\rangle_2) \otimes \cdots \otimes \frac{1}{\sqrt{2}}(|0\rangle_M + |1\rangle_M)$$

$$= \frac{1}{\sqrt{2^M}}\Big(|0\rangle_1 \otimes |0\rangle_2 \otimes \cdots \otimes |0\rangle_M + |0\rangle_1 \otimes |0\rangle_2 \otimes \cdots \otimes |1\rangle_M + \cdots + |1\rangle_1 \otimes |1\rangle_2 \otimes \cdots \otimes |1\rangle_M\Big)$$

上記の式のかっこの中に注目すると、各項は M ビットの二進数で表現できる全ての数と対応していることが分かる。（テンソル積を無視して読むとわかりやすい）

0 から $2^M - 1$ の二進数表現が並んでいるため、上記の式を以下のようにまとめて表現する

$$\frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}}$$

制御ビットにて、2^M 通りの重ね合わせの形を作ることができた。一方、ターゲットビットは恒等演算子により $|\psi\rangle_{\mathcal{N}}$ のまま変化しないため、全体の状態は以下のとおりである。

$$|\psi_1\rangle = \frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}} \otimes |\psi\rangle_{\mathcal{N}}$$

次のステップでは、この重ね合わせ状態を利用して、位相 φ を制御ビットに埋め込む。

ステップ 3：制御ユニタリー U^y を適用

このステップでは、制御ビットの値 y に応じて、ターゲットビットにユニタリー演算子 U を y 回適用する「制御ユニタリー演算」を実行する。

制御ユニタリゲート $U_{\mathcal{M}\otimes\mathcal{N}}$ を以下のように定義する。

$$U_{\mathcal{M}\otimes\mathcal{N}}\Big(|y\rangle_{\mathcal{M}}\otimes|\psi\rangle_{\mathcal{N}}\Big)=|y\rangle_{\mathcal{M}}\otimes U^y|\psi\rangle_{\mathcal{N}}$$

つまり、制御ビットが $|y\rangle$ のとき、ターゲットビットに U を y 回適用するということである。

このユニタリゲートを $|\psi_1\rangle$ に作用させる

$$|\psi_2\rangle=U_{\mathcal{M}\otimes\mathcal{N}}|\psi_1\rangle=\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}U_{\mathcal{M}\otimes\mathcal{N}}\Big(|y\rangle_{\mathcal{M}}\otimes|\psi\rangle_{\mathcal{N}}\Big)$$

$$=\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}|y\rangle_{\mathcal{M}}\otimes U^y|\psi\rangle_{\mathcal{N}}$$

ここで、$|\psi\rangle$ が U の固有状態であるため、以下の式が成り立つ。

$$U|\psi\rangle=e^{2\pi i\varphi}|\psi\rangle$$

したがって、U を y 回適用すると以下のように計算できる。

$$U^y|\psi\rangle=\underbrace{U\cdot U\cdot\cdots\cdot U}_{y\text{回}}|\psi\rangle=(e^{2\pi i\varphi})^y|\psi\rangle=e^{2\pi iy\varphi}|\psi\rangle$$

この関係を使って、$|\psi_2\rangle$ は以下のように書ける。

$$|\psi_2\rangle=\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}|y\rangle_{\mathcal{M}}\otimes U^y|\psi\rangle_{\mathcal{N}}$$

$$=\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}|y\rangle_{\mathcal{M}}\otimes e^{2\pi iy\varphi}|\psi\rangle_{\mathcal{N}}$$

$$=\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}e^{2\pi iy\varphi}|y\rangle_{\mathcal{M}}\otimes|\psi\rangle_{\mathcal{N}}$$

位相 $e^{2\pi iy\varphi}$ は y のみに依存するため、この状態は制御レジスタとターゲットレジスタの直積として書き直せる：

$$|\psi_2\rangle=\Big(\frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}e^{2\pi iy\varphi}|y\rangle_{\mathcal{M}}\Big)\otimes|\psi\rangle_{\mathcal{N}}$$

よって以降は制御レジスタのみを追えばよい。

最後の式では、もともとターゲットビット側 (右側) の「U の固有値」として暗黙的に存在していた位相 φ の情報が、制御ビット側 (左側) で、各 $|y\rangle$ の係数という形で現れたと捉えることができる。この制御ビットの位相 φ を、上手く工夫することで読み出したい。

しかし、このまま制御ビット測定をしても全ての $y = 0, 1, 2, \ldots, 2^M - 1$ が等確率で測定されるだけであり、位相 φ の情報は得られない。

実際の回路実装（controlled-U^{2^k} ゲート）

上記の説明では「U を y 回適用」と書いたが、実際の回路では U を文字通り y 回繰り返すわけではありません。制御ビット $|y\rangle$ を M ビットの二進数として $y = y_{M-1} \cdot 2^{M-1} + \cdots + y_1 \cdot 2 + y_0$ と表すと、指数法則より

$$U^y = U^{y_{M-1} \cdot 2^{M-1}} \cdot U^{y_{M-2} \cdot 2^{M-2}} \cdots U^{y_0}$$

と分解できます。ここで各 $y_k \in \{0, 1\}$ なので以下のように整理できます。

- $y_k = 0$ のとき $\to U^{0 \cdot 2^k} = U^0 = I$（何もしない）
- $y_k = 1$ のとき $\to U^{1 \cdot 2^k} = U^{2^k}$ を適用

したがって、回路には M 個の **controlled-U^{2^k}** ゲート（$k = 0, 1, \ldots, M-1$）を配置し、k 番目の制御ビットが $|1\rangle$ のときだけ U^{2^k} を適用するという方式をとります。

具体例：$M = 3$, $y = 5 = 101_2$ の場合

ビット k	y_k	適用	結果
0	1	$U^{2^0} = U^1$	U^1
1	0	I	何もしない
2	1	$U^{2^2} = U^4$	U^4

合計：$U^1 \cdot I \cdot U^4 = U^{1+4} = U^5$

これにより、ゲート数は y の大きさ（最大 $2^M - 1$）ではなく、制御ビット数 M に比例します。

Shor アルゴリズムでの具体的な実装

Shor で使う $U_{a,n}$ は「a を掛けて n で割った余りを求める」演算でした。($U_{a,n}|x\rangle = |ax \bmod n\rangle$)
愚直に考えると、$U_{a,n}^{2^k}$ は上の操作を何度も作用させることに対応しますが、実際にはそうしません。

$$U_{a,n}^{2^k}|x\rangle = |a^{2^k} x \bmod n\rangle$$

つまり「$a^{2^k} \bmod n$ を掛ける」という **1 回の乗算**であると考えます。

k	掛ける定数（古典で事前計算）
0	$a^1 \bmod n$
1	$a^2 \bmod n$

2	$a^4 \bmod n$
...	...

「量子ゲート自体を二乗する」のではなく、**掛ける定数の値が違う同じ構造の回路**を M 個用意するというわけです。定数 $a^{2^k} \bmod n$ は古典的に繰り返し二乗法で事前計算しておきます。

ステップ 4：制御ビットに逆量子フーリエ変換（逆 QFT）を適用

本ステップでは、制御ビット (左側) への逆量子フーリエ変換によって、位相情報を**測定可能な形**（→ 基底状態の確率分布として現れる形）に変換する。(※逆 QFT は制御ビットにのみ適用する。ターゲットビットは固有状態 $|\psi\rangle$ のまま)。

逆 QFT の定義は以下の通りである。

$$U_{\mathrm{QFT}}^{\dagger}|y\rangle = \frac{1}{\sqrt{2^M}} \sum_{z=0}^{2^M-1} e^{-2\pi i y \frac{z}{2^M}} |z\rangle \tag{3.7}$$

$|\psi_2\rangle$ に逆 QFT（$U_{\mathrm{QFT}}^{\dagger}$）を適用すると以下の通り計算できる

$$U_{\mathrm{QFT}}^{\dagger}\left(\frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} e^{2\pi i y \varphi}|y\rangle\right) = \frac{1}{2^M} \sum_{y=0}^{2^M-1} \sum_{z=0}^{2^M-1} e^{2\pi i y (\varphi - \frac{z}{2^M})}|z\rangle$$

ここで、測定値 z の確率振幅を強調して式を整理すると、以下のように書ける

$$= \frac{1}{2^M} \sum_{z=0}^{2^M-1} \left(\sum_{y=0}^{2^M-1} e^{2\pi i y (\varphi - \frac{z}{2^M})}\right)|z\rangle$$

それでは次に、制御ビットを測定すると、どの z が確率的に得られやすいのかを考える。確率振幅 $A(\theta)$ を、$\theta = \pi\left(\varphi - \frac{z}{2^M}\right)$ として、以下のように書く。

$$A(\theta) = \frac{1}{2^M} \sum_{y=0}^{2^M-1} e^{2iy\theta} \tag{3.8}$$

ケース A：$\theta = 0$ の場合

$\theta = 0$ のとき、$e^{2iy\theta} = e^0 = 1$ となり、確率振幅が最大値 1 をとる

$$A(\theta) = \frac{1}{2^M} \sum_{y=0}^{2^M-1} 1 = \frac{1}{2^M} \cdot 2^M = 1$$

ケース B：$\theta \neq 0$ の場合

一方で、$\theta \neq 0$ のとき、各項が異なる位相を持ち、部分的に打ち消し合う。測定確率は $P(\theta) = |A(\theta)|^2$ で与えられるので、まず $A(\theta)$ を計算してその絶対値を求める。

$$A(\theta) = \frac{1}{2^M} \sum_{y=0}^{2^M-1} e^{2iy\theta} = \frac{1}{2^M}(1 + e^{2i\theta} + e^{2i\cdot 2\theta} + \cdots + e^{2i(2^M-1)\theta}) \tag{3.9}$$

両辺に $e^{2i\theta}$ を掛けると、

$$e^{2i\theta}A(\theta) = \frac{1}{2^M}(e^{2i\theta} + e^{2i\cdot 2\theta} + e^{2i\cdot 3\theta} + \cdots + e^{2i\cdot 2^M\theta}) \tag{3.10}$$

のように書け、式 (3.9) と式 (3.10) の差を取ると以下のように整理できる

$$(1 - e^{2i\theta})A(\theta) = \frac{1}{2^M}(1 - e^{2i\theta\cdot 2^M})$$

$\theta \neq 0$ のとき $e^{2i\theta} \neq 1$ なので、両辺を $(1 - e^{2i\theta})$ で割ることができる

$$A(\theta) = \frac{1}{2^M}\cdot\frac{1 - e^{2i\theta\cdot 2^M}}{1 - e^{2i\theta}} = \frac{1}{2^M}\cdot\frac{e^{i\theta\cdot 2^M}\left(e^{-i\theta\cdot 2^M} - e^{i\theta\cdot 2^M}\right)}{e^{i\theta}\left(e^{-i\theta} - e^{i\theta}\right)} = \frac{1}{2^M}\frac{e^{i\theta\cdot 2^M}\left(-2i\sin\left(2^M\theta\right)\right)}{e^{i\theta}\left(-2i\sin\left(\theta\right)\right)}$$

したがって、確率振幅 $A(\theta)$ の絶対値は以下のように計算できる

$$|A(\theta)| = \frac{1}{2^M}\frac{\left|e^{i\theta\cdot 2^M}\right|\left|-2i\sin\left(2^M\theta\right)\right|}{|e^{i\theta}|\,|-2i\sin\left(\theta\right)|} = \frac{1}{2^M}\frac{\left|\sin\left(2^M\theta\right)\right|}{|\sin\left(\theta\right)|} \tag{3.11}$$

この式の振る舞いは、z が $2^M\varphi$ にどれだけ近いかによって大きく異なる。

z が $2^M\varphi$ から遠い場合

$|\varphi - z/2^M|$ が定数（0 から離れている）のとき、$\theta = \pi(\varphi - z/2^M)$ も定数となるため $|\sin(\theta)|$ は正の定数である。分子の $|\sin(2^M\theta)| \leq 1$ なので、$|A(\theta)| \leq \frac{1}{2^M|\sin(\theta)|}$ となり、確率振幅は M が大きいほど指数的に小さくなる。

z が $2^M\varphi$ に最も近い整数（z_0 とする）の場合

$|\varphi - z_0/2^M| \leq 1/2^{M+1}$ なので $|\theta| = \pi|\varphi - z_0/2^M| \leq \pi/2^{M+1}$ と非常に小さい。このとき $|2^M\theta| \leq \pi/2$ であるから、Jordan の不等式 $|\sin x| \geq (2/\pi)|x|$（$|x| \leq \pi/2$）を分子に適用すると $|\sin(2^M\theta)| \geq (2/\pi)|2^M\theta|$。一方、分母には標準的な不等式 $|\sin\theta| \leq |\theta|$ を適用すると、

$$|A(\theta)| = \frac{|\sin(2^M\theta)|}{2^M|\sin\theta|} \geq \frac{(2/\pi)\cdot 2^M|\theta|}{2^M|\theta|} = \frac{2}{\pi}$$

となり、$|A(\theta)|$ は定数オーダーとなる。**確率振幅は大きな値をとる。**

まとめると、**測定確率は $z \approx 2^M\varphi$（φ に最も近い $z/2^M$）の付近に集中し、そこから離れた z では指数的に小さくなる。**上記の議論から、$2^M\varphi$ に最も近い整数 z_0 が測定される確率について $P(z_0) = |A(\theta)|^2 \geq 4/\pi^2 \approx 0.405$ という下界が得られる（φ が $z/2^M$ と正確に一致する特殊な場合は確率 1）。

コラム：QFT はユニタリ演算子である

一応、QFT の定義式がユニタリ演算子であることを確認します。

QFT は計算基底 $\{|0\rangle, |1\rangle, ..., |2^M - 1\rangle\}$ を以下のように変換します。

$$U_{\mathrm{QFT}}|y\rangle = \frac{1}{\sqrt{2^M}}\sum_{z=0}^{2^M-1} e^{2\pi i y\frac{z}{2^M}}|z\rangle$$

$y \neq y'$ のとき、以下のように書くことができます。

$$\langle U_{\mathrm{QFT}} y | U_{\mathrm{QFT}} y' \rangle = \frac{1}{2^M} \sum_{z=0}^{2^M-1} e^{2\pi i (y'-y) \frac{z}{2^M}} = 0$$

最後の等号は、$\omega = e^{2\pi i (y'-y)/2^M}$ とおくと、$y \neq y'$ のとき $\omega \neq 1$ かつ $\omega^{2^M} = 1$ なので、等比級数の和 $\sum_{z=0}^{2^M-1} \omega^z = \frac{1-\omega^{2^M}}{1-\omega} = 0$ となることを使いました。$y = y'$ のときは $\omega = 1$ なので和は 2^M となり、正規化係数で割って 1 となります。
したがって QFT は正規直交基底を正規直交基底に移すので、ユニタリ演算子です。ユニタリ演算子は量子回路として物理的に実装可能であり、QFT の場合はアダマールゲートと制御位相ゲートを組み合わせて $O(M^2)$ 個のゲートで構成できます（詳細は Nielsen & Chuang [2] 第 5 章を参照してください）。

ステップ 5：位相の測定

以上より、制御ビットを測定すると、高確率で $\theta = 0 \to z \approx 2^M \varphi$ となる z が得られる。

したがって、測定値 z を用いて、U の位相 φ は以下のように求められる

$$\varphi \approx \frac{z}{2^M}$$

以上より、ユニタリ演算子 U の固有値 $e^{2\pi i \varphi}$ の位相 φ を得られた。

QPE はあくまで一般的なアルゴリズムであり、ここまでで計算量については議論していない。QPE が「速い」かどうかは、対象となるユニタリ演算子 U を効率的に実装できるかに依存する。

次節では、QPE を位数発見問題に適用する。鍵となるのは、モジュラー乗算演算子 $U_{a,n}$ が多項式個のゲートで実装できることである。これが Shor アルゴリズムの計算量を多項式時間に抑える本質である。

3.4　Shor のアルゴリズム

ここまで説明した量子位相推定は、任意のユニタリ演算子に適用できる一般的なアルゴリズムである。Shor のアルゴリズムは、この量子位相推定をモジュラー乗算演算子という特殊なユニタリ演算子に適用した応用例である。

本章では、QPE に代入するモジュラー乗算演算子を導入し、Shor アルゴリズムの具体的な手順を説明する。なお、理想的な量子コンピュータを前提とした議論であり、現実の技術的課題は 3.5.6 節で扱う。

3.4.1　モジュラー乗算演算子

モジュラー乗算演算子の定義

Shor アルゴリズムでは、QPE に以下で定義される**モジュラー乗算演算子** $U_{a,n}$ を代入する。

$$U_{a,n}|x\rangle = \begin{cases} |ax \bmod n\rangle & (0 \leq x < n) \\ |x\rangle & (n \leq x < 2^N) \end{cases}$$

ここで、$N = \lceil \log_2 n \rceil$ はターゲットレジスタのビット数、n は因数分解したい合成数で、a は $1 < a < n$ かつ $\gcd(a, n) = 1$ を満たすランダムに選んだ整数である。$0 \leq x < n$ の範囲では x に a を 1 回掛けて n で割った余りを求め（乗算）、$x \geq n$ の範囲では恒等写像とする。量子回路は 2^N 次元のヒルベルト空間上で動作するため、$x \geq n$ の基底状態への作用も定義する必要があり、この拡張により $U_{a,n}$ は全計算基底上の置換となる。

モジュラー乗算演算子の具体例（$n = 15, a = 7$）

- $U_{7,15}|1\rangle = |7 \times 1 \bmod 15\rangle = |7\rangle$
- $U_{7,15}|2\rangle = |7 \times 2 \bmod 15\rangle = |14\rangle$
- $U_{7,15}|7\rangle = |7 \times 7 \bmod 15\rangle = |49 \bmod 15\rangle = |4\rangle$
- $U_{7,15}|4\rangle = |7 \times 4 \bmod 15\rangle = |28 \bmod 15\rangle = |13\rangle$

モジュラー乗算 vs モジュラー累乗

- **モジュラー乗算演算子** $U_{a,n}$：$|x\rangle \mapsto |ax \bmod n\rangle$（$a$ を 1 回掛ける）
- **モジュラー累乗**：$U_{a,n}$ を y 回適用すること $\rightarrow |x\rangle \mapsto |a^y x \bmod n\rangle$

Shor アルゴリズムでは、QPE の制御ユニタリとして $U_{a,n}^y$（y 回適用）を使うため、ステップ 3 では「制御モジュラー累乗」と呼びます。

コラム：$U_{a,n}$ のユニタリ性

$\gcd(a, n) = 1$ より、写像 $x \mapsto ax \bmod n$ は $\{0, 1, ..., n-1\}$ 上の全単射です。$x \geq n$ では恒等写像としたので、全体として $\{0, 1, ..., 2^N - 1\}$ 上の置換（全単射）になります。
$U_{a,n}$ は計算基底 $|x\rangle$ を別の計算基底に 1 対 1 で写します。異なる計算基底は異なる計算基底に写り、計算基底は正規直交なので、内積が保存されます。したがって $U_{a,n}$ はユニタリです。

モジュラー乗算演算子と位数の関係

この演算子の重要な性質は、位数 r（$a^r \equiv 1 \pmod{n}$ を満たす最小の正整数）回適用すると単位演算子になることである

$$U_{a,n}^r = I$$

例えば、$|x\rangle = |1\rangle$ として、繰り返し $U_{a,n}$ を k 回作用させると以下の式が成り立つ

$$(U_{a,n})^k |1\rangle = |a^k \bmod n\rangle$$

ここで、位数 r の定義を思い出せば以下の式が成り立つ

$$a^r \equiv 1 \pmod{n}$$

$$(U_{a,n})^r |1\rangle = |1\rangle$$

このように、モジュラー乗算演算子 $U_{a,n}$ を $|1\rangle$ に、位数 r 回作用させると、$|1\rangle$ に戻るという周期構造をもつことがわかる。この周期構造こそが、量子位相推定で抽出したい情報である。

位相と位数にはどんな関係があるのか？

$|1\rangle$ に $U_{a,n}$ を繰り返し適用すると、$\{|1\rangle, |a\rangle, |a^2\rangle, ..., |a^{r-1}\rangle\}$（すべて $\mathrm{mod} n$）という r 個の状態を巡回する。これらが張る r 次元部分空間において、$U_{a,n}$ の固有状態は以下のように明示的に書ける。

$$|\psi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^k \bmod n\rangle \quad (s = 0, 1, ..., r-1)$$

試しに、この固有状態に $U_{a,n}$ を作用させる。$U_{a,n}|a^k\rangle = |a^{k+1}\rangle$ であり、$|a^r\rangle = |a^0\rangle = |1\rangle$（周期性）なので、

$$U_{a,n}|\psi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^{k+1}\rangle = e^{2\pi i \frac{s}{r}} \cdot \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^k\rangle = e^{2\pi i \frac{s}{r}} |\psi_s\rangle$$

となり、固有値は $e^{2\pi i \frac{s}{r}}$ であることが分かる。

これらの固有状態 $\{|\psi_0\rangle, ..., |\psi_{r-1}\rangle\}$ は正規直交基底をなす。$|1\rangle$ はこの基底で展開できる。

$$|1\rangle = \sum_{s=0}^{r-1} \langle\psi_s|1\rangle |\psi_s\rangle$$

係数は $\langle\psi_s|1\rangle = \frac{1}{\sqrt{r}}$（定義から $k = 0$ の項のみ寄与）なので、

$$|1\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\psi_s\rangle$$

QPE で $|1\rangle$ を入力すると、確率 $\frac{1}{r}$ で各固有状態 $|\psi_s\rangle$ に対応する位相 s/r に対応するピーク位置 $z \approx 2^M \cdot s/r$ が高確率で得られる。どの s が得られるかは量子測定の確率に従うが、分母は常に r（または r の約数）なので、位数を抽出できる。

以上より、**モジュラー乗算演算子の「位相を測定する」ことで、「位数を求める」ことができるという論理が導かれた。**

3.4.2　Shor のアルゴリズムの手順

前節で説明した QPE の手順に沿って、Shor のアルゴリズム特有の部分に注目して議論していきたい。

ステップ 1：初期状態を準備

量子位相推定と同様に、以下の 2 種類の量子ビットを用意する。

① M 個の制御ビット（位相情報を読み取るためのビット列）

M 個の量子ビット $|0\rangle_{\mathcal{M}} = |0\rangle_1 \otimes |0\rangle_2 \otimes ... \otimes |0\rangle_M$ を用意する。M を大きくすれば、測定精度は向上するが、測定時間が増える。（詳細はコラムで後述）。

② N 量子ビットのターゲットレジスタ（モジュラー累乗演算を行うためのビット列）

Shor アルゴリズムでは、ターゲットレジスタ $\mathcal{N}$（N 量子ビット）を $|1\rangle_{\mathcal{N}}$ で初期化する。ここで $|1\rangle_{\mathcal{N}}$ は、整数 1 を N ビットの二進数で表現した状態である。

$$|1\rangle_{\mathcal{N}} = |0\rangle_1 \otimes |0\rangle_2 \otimes ... \otimes |0\rangle_{N-1} \otimes |1\rangle_N = |0...01\rangle$$

（※ $|1\rangle_1 \otimes |1\rangle_2 \otimes ... \otimes |1\rangle_N$ ではないことに注意）

注意：$|1\rangle_{\mathcal{N}}$ はモジュラー乗算演算子の固有状態ではない

一般的な QPE の説明では、固有状態 $|\psi\rangle$ を使うことで特定の位相 φ の狙い撃ちをした。しかし Shor アルゴリズムでは、$U_{a,n}$ の固有状態を明示的に作るには位数 r を知る必要がある——しかし、これはまさに今求めたい値そのものであり、循環してしまう。

そこで、ここでは固有状態の代わりに $|1\rangle$ を使う。3.4.1 節で述べた通り、$U_{a,n}$ は r 個の正規直交な固有状態 $\{|\psi_0\rangle, |\psi_1\rangle, ..., |\psi_{r-1}\rangle\}$ を持ち、$|1\rangle$ はこの基底で展開できる。

$$|1\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\psi_s\rangle$$

もちろん、$|1\rangle$ は固有状態ではないため、特定の位相を狙い撃ちできなくなる。QPE を実行すると、r 個の固有状態に対応する位相 s/r（$s = 0, 1, \ldots, r-1$）に対応するピーク位置 $z \approx 2^M \cdot s/r$ が高確率で得られる。連分数展開で分母を取り出すと $r/\gcd(s,r)$ となり、$\gcd(s,r) = 1$ のとき位数 r そのものが得られる。それ以外の場合は r の約数となるため、倍数を試すか測定をやり直す（詳細は後述）。

★ターゲットレジスタの量子ビット数 N の条件

モジュラー乗算演算子 $U_{a,n}$ は n 次元空間に作用するため、ターゲットレジスタの量子ビット数 N は $2^N \geq n$ を満たす必要がある。

$$2^N \geq n$$

具体例

- $n = 15$ **の場合**
  $2^4 = 16 > 15$ なので $N = 4$ で十分
- $n = 21$ **の場合**
  $2^5 = 32 > 21$ なので $N = 5$ で十分
- **RSA-2048（$n \approx 2^{2048}$）の場合**
  $N = 2048$ が必要

以上をまとめると、初期状態は以下の通りである。

$$|\psi_0\rangle = |0\rangle_{\mathcal{M}} \otimes |1\rangle_{\mathcal{N}} \tag{3.12}$$

ステップ 2：アダマールゲートの作用（QPE の時と同じ）

本ステップでは、初期状態 $|\psi_0\rangle = |0\rangle_{\mathcal{M}} \otimes |1\rangle_{\mathcal{N}}$ の制御ビット（$\mathcal{M}$）にのみアダマールゲートを作用させ、ターゲットビット（$\mathcal{N}$）には恒等演算子 I（何もしない）を作用させる。

$$\left(\underbrace{U_H \otimes U_H \otimes \cdots \otimes U_H}_{M\text{個}}\right) \otimes \left(\underbrace{I \otimes I \otimes \cdots \otimes I}_{N\text{個}}\right) |0\rangle_{\mathcal{M}} \otimes |1\rangle_{\mathcal{N}} \tag{3.13}$$

QPE の際に計算した通り、制御ビット部分に注目すると以下のように書くことができる

$$\frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}} \tag{3.14}$$

ターゲットビットは恒等演算子により $|1\rangle_{\mathcal{N}}$ のまま変化しないため、全体の状態は以下のとおりである。

$$|\psi_1\rangle = \frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}} \otimes |1\rangle_{\mathcal{N}} \tag{3.15}$$

これが量子コンピュータの**並列性の源**である。しかし、この並列性だけでは Shor アルゴリズムの優位性は説明できないことに注意してほしい。よく重ね合わせの式を見せて、古典コンピュータより量子コンピュータの方が効率的に答えを求められるかのような説明を見るときがあるが、それは誤解である。結局このまま測定しても、得られる情報は一つの項の値にすぎない。次のステップ以降で、量子干渉と連分数展開という 2 つの工夫により、この並列計算から周期情報を抽出する仕組みを見ていく。

ステップ 3：制御モジュラー累乗演算

本ステップでは、3.4.1 節で説明した**モジュラー乗算演算子** $U_{a,n}$ を QPE のユニタリ演算子として使用する。

3.4.1 節で見たように、$U_{a,n}$ の固有値は $e^{2\pi i \frac{s}{r}}$ の形をしており、位相の分母に位数 r が現れる。これこそが Shor アルゴリズムでこの演算子を採用する理由である。

a の選択（ユーザーの意思決定ポイント）

ここで、$1 < a < n$ の範囲から自然数 a をランダムに選択する。

- $\gcd(a, n) = 1$（互いに素）であることを確認する
- もし $\gcd(a, n) \neq 1$ なら、その gcd が n の非自明な約数なので、量子計算なしで因数分解完了（極めて稀）

a が決まれば、制御ビットの値 y に応じて $U_{a,n}$ を y 回適用する**制御モジュラー累乗演算**を以下のように定義できる

$$U^{(a)}_{\mathcal{M}\otimes\mathcal{N}}\Big(|y\rangle_{\mathcal{M}} \otimes |x\rangle_{\mathcal{N}}\Big) = |y\rangle_{\mathcal{M}} \otimes |x \cdot a^y \bmod n\rangle_{\mathcal{N}} \tag{3.16}$$

ここでは $0 \leq x < n$ の部分空間に議論を限定している（$x \geq n$ については 3.4 節冒頭の $U_{a,n}$ の

定義通り恒等写像とする)。アルゴリズムが実際に訪れるのはこの部分空間であり、以下の議論に影響しない。このユニタリゲートを $|\psi_1\rangle$ に作用させる。

$$|\psi_2\rangle = U^{(a)}_{\mathcal{M}\otimes\mathcal{N}}|\psi_1\rangle = \frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1} U^{(a)}_{\mathcal{M}\otimes\mathcal{N}}\Big(|y\rangle_{\mathcal{M}}\otimes|1\rangle_{\mathcal{N}}\Big)$$

$$= \frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}|y\rangle_{\mathcal{M}}\otimes|1\cdot a^y \bmod n\rangle_{\mathcal{N}} = \frac{1}{\sqrt{2^M}}\sum_{y=0}^{2^M-1}|y\rangle_{\mathcal{M}}\otimes|a^y \bmod n\rangle_{\mathcal{N}}$$

さらに、理解のために、この式を具体的に書き下して見る。

$$|\psi_2\rangle = \frac{1}{\sqrt{2^M}}\Big(|0\rangle_{\mathcal{M}}\otimes|a^0 \bmod n\rangle_{\mathcal{N}} + |1\rangle_{\mathcal{M}}\otimes|a^1 \bmod n\rangle_{\mathcal{N}} + |2\rangle_{\mathcal{M}}\otimes|a^2 \bmod n\rangle_{\mathcal{N}} + \cdots\Big)$$

たとえば $n = 15, a = 7$ の場合（$r = 4$)、各項は以下のように書き下すことができる。

$$\begin{aligned}|\psi_2\rangle = \frac{1}{\sqrt{2^M}}\Big(&|0\rangle\otimes|1\rangle + |1\rangle\otimes|7\rangle + |2\rangle\otimes|4\rangle + |3\rangle\otimes|13\rangle \\ &+ |4\rangle\otimes|1\rangle + |5\rangle\otimes|7\rangle + |6\rangle\otimes|4\rangle + |7\rangle\otimes|13\rangle + \cdots\Big)_{\mathcal{M}\otimes\mathcal{N}}\end{aligned}$$

ターゲットビット側 (右側) を見ると、$|1\rangle, |7\rangle, |4\rangle, |13\rangle$ という 4 つの値が周期的に繰り返されている。

この構造をターゲット値ごとにまとめると、以下のように書ける。

（以下では簡単のため $r|2^M$ の場合の式形を示す。一般の場合の扱いは後の注記を参照。）

$$\begin{aligned}|\psi_2\rangle = \frac{1}{\sqrt{2^M}}\Big[\ &\big(|0\rangle + |4\rangle + |8\rangle + \cdots\big)_{\mathcal{M}}\otimes|1\rangle_{\mathcal{N}} \\ &+ \big(|1\rangle + |5\rangle + |9\rangle + \cdots\big)_{\mathcal{M}}\otimes|7\rangle_{\mathcal{N}} \\ &+ \big(|2\rangle + |6\rangle + |10\rangle + \cdots\big)_{\mathcal{M}}\otimes|4\rangle_{\mathcal{N}} \\ &+ \big(|3\rangle + |7\rangle + |11\rangle + \cdots\big)_{\mathcal{M}}\otimes|13\rangle_{\mathcal{N}}\ \Big]\end{aligned}$$

位数の定義から $a^r \equiv 1 \pmod n$ なので r ごとに同じ値に戻る。したがって、ターゲットビット側（右側）に周期 r が現れるのは当然である。

一方で、**重要なのは、制御ビット側（左側）にも周期 r の構造が現れていること**である。各列を見ると、$|0\rangle, |4\rangle, |8\rangle, ...$、$|1\rangle, |5\rangle, |9\rangle, ...$ のように、**間隔 $r = 4$ で並んでいる**。

これが制御モジュラー累乗演算のポイントであり、次のステップで説明する**逆量子フーリエ変換はこの制御ビット側の周期 r を検出**する演算である。

コラム：$|\psi_2\rangle$ はエンタングルメント状態

ステップ 2 の完了時点では、状態は積の形でした。

$$|\psi_1\rangle = \frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}} \otimes |1\rangle_{\mathcal{N}}$$

制御レジスタとターゲットレジスタは独立であり、$|\psi\rangle_{\mathcal{M}} \otimes |\phi\rangle_{\mathcal{N}}$ の形に分解できます。
しかしステップ 3 で制御モジュラー累乗を適用すると:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} |y\rangle_{\mathcal{M}} \otimes |a^y \bmod n\rangle_{\mathcal{N}}$$

各制御値 $|y\rangle$ が特定のターゲット値 $|a^y \bmod n\rangle$ と**ペア**になり、全体の状態が $|\psi\rangle_{\mathcal{M}} \otimes |\phi\rangle_{\mathcal{N}}$ の形に分解できなくなります。これが**エンタングルメント（量子もつれ）**です。
この相関により、ターゲットビット側の周期構造が制御ビット側にも現れています。
エンタングルメントと古典的な相関の違い
「片方を測定すると相手が決まる」という相関は、古典的にも存在します。例えば、赤い靴下と青い靴下を別々の箱に入れて離れた場所に送り、片方を開けたら相手の色が分かる——これは古典的な相関であり、靴下の色は箱に入れた時点で確定しています。
エンタングルメントが古典相関と本質的に異なるのは、**測定する基底（どの方向で測るか）を変えても相関が存在する**点です。古典の靴下は「赤か青か」という 1 つの基底でしか相関がありませんが、エンタングルした量子ビットはどんな基底で測定しても（$|0\rangle/|1\rangle$ でも、$|+\rangle/|-\rangle$ でも）相関が現れます。この性質はベルの不等式として定式化され、実験的にも検証されています。
Shor のアルゴリズムでは、ステップ 3 でエンタングルメントが生じ、ステップ 4（逆 QFT）でその量子的な相関を利用して位相情報を読み出します。エンタングルメントは量子計算における重要な計算資源の一つです。

ステップ 4：制御ビットに逆量子フーリエ変換（逆 QFT）を適用

ここからの議論で密度行列を用いる: 以下のステップ 4 では、制御レジスタの測定結果を計算するため、密度行列 $\rho = |\psi\rangle\langle\psi|$ と部分トレース $\mathrm{Tr}_{\mathcal{N}}(\cdot)$ を用います。これは「複合系の一部だけを測定するときの確率分布を計算するための標準的な道具」であり、QPE の章では直積状態を扱っていたため不要でしたが、Shor アルゴリズムでは制御ビットとターゲットビットがエンタングル状態になるため必要になります。詳細は量子情報の標準的な教科書（Nielsen & Chuang [2] など）を参照してください。

本ステップでは、ステップ 3 で制御ビット側 (左側) に現れた**等間隔構造**を「測定可能な形」に変換する。

逆 QFT の適用

逆量子フーリエ変換 $U_{\mathrm{QFT}}^{\dagger}$ の定義は以下の通りである。

$$U_{\mathrm{QFT}}^{\dagger}|y\rangle = \frac{1}{\sqrt{2^M}} \sum_{z=0}^{2^M-1} e^{-2\pi i y \frac{z}{2^M}} |z\rangle \tag{3.17}$$

ステップ 3 の結果 $|\psi_2\rangle$ に逆 QFT を適用する。制御ビット (左側) にのみ作用し、ターゲットビッ

ト (右側) には何もしない。

$$|\psi_3\rangle = (U_{\mathrm{QFT}}^\dagger \otimes I)|\psi_2\rangle = \frac{1}{\sqrt{2^M}} \sum_{y=0}^{2^M-1} (U_{\mathrm{QFT}}^\dagger |y\rangle_{\mathcal{M}}) \otimes |a^y \bmod n\rangle_{\mathcal{N}}$$

式 (3.17) を代入すると、以下のように書ける。

$$= \frac{1}{2^M} \sum_{y=0}^{2^M-1} \sum_{z=0}^{2^M-1} e^{-2\pi i y \frac{z}{2^M}} |z\rangle_{\mathcal{M}} \otimes |a^y \bmod n\rangle_{\mathcal{N}}$$

見やすいように整理する。

$$|\psi_3\rangle = \frac{1}{2^M} \sum_{z=0}^{2^M-1} |z\rangle_{\mathcal{M}} \otimes \left(\sum_{y=0}^{2^M-1} e^{-2\pi i y \frac{z}{2^M}} |a^y \bmod n\rangle_{\mathcal{N}} \right) \tag{3.18}$$

制御ビットの測定確率を求める

Shor のアルゴリズムでは、制御ビットのみを測定し、ターゲットビットは測定せずに破棄する。

QPE の説明時は直積状態だったため、単にターゲットビットだけに注目すればよかった。一方で、式 (3.18) のエンタングルした状態 $|\psi_3\rangle$ の一部（ターゲットビット）を無視して、残り（制御ビット）だけを測定する場合、部分トレースを用いて制御ビットの縮約密度行列を計算する必要がある。

複合系の密度行列は $\rho = |\psi_3\rangle\langle\psi_3|$ と書ける。次に、ターゲットビットをトレースアウトした縮約密度行列は以下のように書ける。

$$\rho_{\mathcal{M}} = \mathrm{Tr}_{\mathcal{N}}(|\psi_3\rangle\langle\psi_3|) \tag{3.19}$$

この時、制御ビットで $|z\rangle$ を測定する確率は以下で計算できるので、

$$P(z) = \langle z|\rho_{\mathcal{M}}|z\rangle \tag{3.20}$$

式 (3.18) を式 (3.19) に代入して計算してみる。まず、複合系 $|\psi_3\rangle\langle\psi_3|$ を展開すると、以下のように整理できる。

$$|\psi_3\rangle\langle\psi_3| = \frac{1}{2^{2M}} \sum_{z',z''} |z'\rangle\langle z''|_{\mathcal{M}} \otimes \left(\sum_{y,y'} e^{-2\pi i (y \frac{z'}{2^M} - y' \frac{z''}{2^M})} |a^y \bmod n\rangle\langle a^{y'} \bmod n|_{\mathcal{N}} \right)$$

それでは、制御ビット側の縮約密度行列を得るために、ターゲットビット $\mathcal{N}$ をトレースアウトする。

$$\rho_{\mathcal{M}} = \frac{1}{2^{2M}} \sum_{z',z''} |z'\rangle\langle z''|_{\mathcal{M}} \sum_{y,y'} e^{-2\pi i (y \frac{z'}{2^M} - y' \frac{z''}{2^M})} \langle a^{y'} \bmod n|a^y \bmod n\rangle$$

我々が知りたいのはある z が測定される確率である。式 (3.20) で記載したとおり、$P(z) = \langle z|\rho_{\mathcal{M}}|z\rangle$ なので、$z' = z'' = z$ (z', z'' はダミー変数で、z は測定値である) の項だけが残り、ある z が測定される確率は以下のように書くことができる。

$$P(z) = \frac{1}{2^{2M}} \sum_{y,y'=0}^{2^M-1} e^{-2\pi i(y-y')\frac{z}{2^M}} \langle a^{y'} \bmod n | a^y \bmod n \rangle \tag{3.21}$$

それでは、式 (3.21) の内積 $\langle a^{y'} \bmod n | a^y \bmod n \rangle$ を評価する。

$|a^y \bmod n\rangle$ はターゲットレジスタ $\mathcal{N}$ の計算基底状態である（3.3 節 ステップ 1 のコラム「計算基底と正規直交性」参照)。計算基底は正規直交基底をなすので、$a^y \bmod n = a^{y'} \bmod n$ のとき 1、それ以外は 0 である。では、$a^y \bmod n = a^{y'} \bmod n$ を満たす条件を考える。位数の定義（式 (3.1)）より $a^r \equiv 1 \pmod n$ なので、$a^y \bmod n$ は位数 r の周期構造を持つ。

$$\begin{aligned}
a^0 \bmod n &= a^r \bmod n = a^{2r} \bmod n = \cdots \\
a^1 \bmod n &= a^{1+r} \bmod n = a^{1+2r} \bmod n = \cdots \\
&\vdots \\
a^{r-1} \bmod n &= a^{r-1+r} \bmod n = a^{r-1+2r} \bmod n = \cdots
\end{aligned}$$

一般に、$a^{j+kr} \bmod n = a^j \bmod n$ である（k は任意の非負整数)。つまり、y を r で割った余りが同じであれば、$a^y \bmod n = a^{y'} \bmod n$ を満たす。そこで、y を r で割った商を k、余りを j として以下のように表す。

注：本来 k の上限は「$2^M/r$ を超えない最大の整数」だが、表記の簡略化のため r が 2^M を割り切るものとして書く。結論は一般の場合も同様に成り立つ。

$$\begin{aligned}
y &= j + kr \quad (j \in \{0, \ldots, r-1\},\ k \in \{0, \ldots, \tfrac{2^M}{r} - 1\}) \\
y' &= j' + k'r \quad (j' \in \{0, \ldots, r-1\},\ k' \in \{0, \ldots, \tfrac{2^M}{r} - 1\})
\end{aligned}$$

上記の変数変換を式 (3.21) に代入すると、以下のように書くことができる。

$$P(z) = \frac{1}{2^{2M}} \sum_{j,j'=0}^{r-1} \sum_{k,k'=0}^{\frac{2^M}{r}-1} e^{-2\pi i((j+kr)-(j'+k'r))\frac{z}{2^M}} \langle a^{j'} \bmod n | a^j \bmod n \rangle$$

したがって、$j = j'$ の項だけが残る。このとき、$(j + kr) - (j' + k'r) = (k - k')r$ となるので以下のようにまとめることができる。

$$= \frac{1}{2^{2M}} \sum_{j=0}^{r-1} \sum_{k,k'=0}^{\frac{2^M}{r}-1} e^{-2\pi i(k-k')r\frac{z}{2^M}} = \frac{r}{2^{2M}} \left| \sum_{k=0}^{\frac{2^M}{r}-1} e^{-2\pi i k r \frac{z}{2^M}} \right|^2$$

ここで、$S(z)$ を以下のように定義する。(QPE の時の確率振幅に対応する)

$$S(z) = \frac{\sqrt{r}}{2^M} \sum_{k=0}^{\frac{2^M}{r}-1} e^{-2\pi i k r \frac{z}{2^M}} \tag{3.22}$$

測定確率は、以下のように書ける。

$$P(z) = |S(z)|^2 \tag{3.23}$$

$S(z)$ の計算 (QPE の確率振幅の計算と同じ)

最終的にターゲットビットの測定の確率を決めるのは式 (3.22) で定義した $S(z)$ だけであることが分かった。

QPE と同様に、$\theta = \pi \cdot \frac{rz}{2^M}$ とおくと、

$$S(\theta) = \frac{\sqrt{r}}{2^M} \sum_{k=0}^{\frac{2^M}{r}-1} e^{-2ik\theta} \tag{3.22'}$$

$S(\theta)$ の和の部分は等比級数なので、以下のようにまとめられる。

$$\sum_{k=0}^{\frac{2^M}{r}-1} e^{-2ik\theta} = \begin{cases} \dfrac{2^M}{r} & (\theta \in \pi\mathbb{Z}) \\ \dfrac{1 - e^{-2i\cdot(\frac{2^M}{r})\theta}}{1 - e^{-2i\theta}} & (\theta \notin \pi\mathbb{Z}) \end{cases}$$

ここで $\theta = \pi \cdot rz/2^M$ は $z = s \cdot 2^M/r$（$s = 0, 1, \ldots, r-1$）のとき $\theta = \pi s \in \pi\mathbb{Z}$ となり、このとき $e^{-2ik\theta} = e^{-2ik\pi s} = 1$ なので、和は最大値 $2^M/r$ をとる。それ以外の z では位相がずれて打ち消し合い、和は小さくなる。

したがって、$z = s \cdot 2^M/r$ のとき $S(z)$, $P(z)$ は以下のように計算できる。

$$S(z) = \frac{\sqrt{r}}{2^M} \cdot \frac{2^M}{r} = \frac{1}{\sqrt{r}} \quad \Rightarrow \quad P(z) = |S(z)|^2 = \frac{1}{r}$$

したがって、**理想的には、r 個のピーク（$z = 0, \frac{2^M}{r}, \frac{2\cdot 2^M}{r}, \ldots, \frac{(r-1)\cdot 2^M}{r}$）がそれぞれ確率 $\frac{1}{r}$ で測定される。**

具体例 1（$n = 15, a = 7, r = 4, M = 10$）→ r が 2^M を割り切る場合

$2^M = 1024$、$\frac{2^M}{r} = 256$（整数）なので、ピークはちょうど整数位置に現れる。

s	$s \cdot 2^M/r$（理論値）	実際に測定される z	確率
0	0	0	1/4
1	256	256	1/4
2	512	512	1/4
3	768	768	1/4

この場合、z の測定から容易に $r = 4$ が求められる。

具体例 2（$n = 21, a = 2, r = 6, M = 10$）→ r が 2^M を割り切らない場合

$2^M = 1024$、$\frac{2^M}{r} = 170.67...$（整数でない）なので、ピーク位置は近似値になる。なお、r が 2^M を割り切らないため、以下の表に示す各確率は厳密値ではなく近似値である（ピーク値は付近の整数に分散する）。

s	$s \cdot 2^M/r$（理論値）	実際に測定される z（付近の整数）	確率（近似値）
0	0	0	$\approx 1/6$
1	170.67	170 または 171	$\approx 1/6$
2	341.33	341 または 342	$\approx 1/6$
3	512	512	$\approx 1/6$
4	682.67	682 または 683	$\approx 1/6$
5	853.33	853 または 854	$\approx 1/6$

一方で、このように一般には $\frac{z}{2^M} \approx \frac{s}{r}$ という近似関係になる。この場合、次節で議論する連分数展開という方法で、z の測定から正確な r を復元する。

ステップ 5：測定と連分数展開による位数の抽出

制御ビットを測定すると、整数 z が得られる。この z から位数 r を求めたい。

測定値と位相の関係

ステップ 4 で見たように、高確率で測定される z は以下を満たす。

$$z \approx \frac{2^M s}{r} \quad (s = 0, 1, ..., r-1 \text{ のいずれか}) \tag{3.24}$$

ここで s は量子測定由来でランダムに決まる。

もし具体例 1 のように、$z = \frac{2^M s}{r}$ と等号が成り立つなら、$\frac{z}{2^M}$ を約分するだけで r が求まる。しかし M は有限なので、測定結果 z は 2^M 通りの離散値しかとれず、ピーク位置と一致する保証はない。そのため近似（$\approx$）になる。

そこで、「$\frac{z}{2^M}$ に最も近い、分母が小さい有理数」を求める手法として**連分数展開**を用いる。$s \neq 0$ であれば分母から r の約数が抽出でき、さらに $\gcd(s, r) = 1$ ならその分母がそのまま r となる（詳細は約分の問題コラムを参照）。

ただし $s = 0$ の場合は $z \approx 0$ となり、$\frac{0}{2^M} = 0$ からは r の情報が得られない。この場合は測定をやり直す。$s = 0$ が出る確率は約 $\frac{1}{r}$ なので、r が大きければ稀である。

コラム：M を増やせば近似でなくなるのか？

「量子ビット数 M を増やせば、$z = \frac{2^M s}{r}$ と等号が成り立つようになるのでは？」と思うかもしれません。

筋はとても良いですが、M が有限である限り、測定結果は 2^M 通りの離散値しかとれません。$\frac{2^M s}{r}$ がこれら離散値のどれかに一致するためには r が 2^M を割り切る必要がありますが、2^M は 2 のべき乗なので、r が 2 のべき乗でない限り（例：$r = 6$）どれだけ M を大きくしても一致することはありません。

理論的には $M \to \infty$ の極限で離散値の集合は連続区間 $[0, 1)$ に収束し、任意の $\frac{s}{r}$ と一致する値が存在するようになります。しかし無限個の量子ビットは使えないので、実用上は近似を受け入れ、連分数展開で r を復元することになります。

連分数展開のアルゴリズム

ここでは、数論において知られている連分数の定理を用いて、制御ビットの測定値 z から位数 r を求める。

まず初めに、連分数展開と収束分数を紹介する。

記号の注意：本節では連分数の分子・分母を p, q で表します。RSA 暗号の素数 p, q とは別物ですが、連分数の文脈では慣例的にこの記号を用いるため、以下では連分数の分子・分母の意味で使用します。

任意の実数 x は以下のように、連分数展開で表現できる。

$$x = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\cdots}}} = [a_0; a_1, a_2, \ldots]$$

この展開を途中で打ち切って得られる分数を**収束分数**と呼ぶ。

$$\frac{p_0}{q_0} = a_0, \quad \frac{p_1}{q_1} = a_0 + \frac{1}{a_1}, \quad \frac{p_2}{q_2} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2}}, \quad \ldots$$

連分数の定理

不等式 $\left| x - \frac{p}{q} \right| < \frac{1}{2q^2}$ を満たす既約分数 $\frac{p}{q}$ は、x の連分数展開の収束分数に必ず現れる。

上記の定理を我々の設定にあてはめると、以下の対応があることが分かる。

- $x = \frac{z}{2^M}$（測定から得られる値）
- $\frac{p}{q} = \frac{s}{r}$（知りたい値）

したがって、$\left| \frac{z}{2^M} - \frac{s}{r} \right| < \frac{1}{2r^2}$ を満たす $\frac{s}{r}$ は、$\frac{z}{2^M}$ の連分数展開の収束分数に必ず現れる。上記の不等式は、制御ビットの個数 M を $M \geq 2\lceil \log_2 n \rceil$ と設定していれば満たすことができる (コラムを参照)

定理は「収束分数のどれかに現れる」と言うだけで、何番目かは分からない。そこで実際の手順では、収束分数を順番に計算し、各分母 r' に対して $a^{r'} \equiv 1 \pmod{n}$ を検証する。

ここで、連分数の定理より既約分数 $\frac{s/\gcd(s,r)}{r/\gcd(s,r)}$ が収束分数として現れるため、その分母 $r/\gcd(s,r)$ ($\leq r$) が検証対象となる。$\gcd(s,r) = 1$ のときは分母がそのまま r となり、検証 $a^r \equiv 1 \pmod{n}$ が成り立つ。一方 $\gcd(s,r) > 1$ のときは分母が r の真の約数となり、位数の最小性から $a^{r/\gcd(s,r)} \not\equiv 1 \pmod{n}$ なので検証に失敗する。この場合は倍数を試すか測定をやり直す（詳細は約分の問題コラムを参照)。

したがって、検証に成功した時点での r' は常に $r' = r$（位数そのもの）である。

なお、位数 r を見つけるのは難しいので、量子コンピュータを使うが、与えられた r' に対して $a^{r'} \equiv 1 \pmod{n}$ を「検証する」のは古典コンピュータで容易である（繰り返し二乗法で $O(\log r')$

回の乗算)。

コラム：$M \geq 2\lceil \log_2 n \rceil$ の導出

z は真のピーク $\frac{2^M s}{r}$ に最も近い整数なので、誤差は最大でも $\frac{1}{2}$ となります。

$$\left|\frac{z}{2^M} - \frac{s}{r}\right| \leq \frac{1}{2 \cdot 2^M} = \frac{1}{2^{M+1}}$$

定理の条件を満たすには、$\frac{1}{2^{M+1}} < \frac{1}{2r^2}$ が必要となります。以下のように整理することができます。

$$2r^2 < 2^{M+1} \quad \Rightarrow \quad r^2 < 2^M \quad \Rightarrow \quad r < 2^{M/2}$$

条件は $r < 2^{M/2}$ ですが、r は事前に分かりません。そこで、r は a の位数なので $r < n$ が成り立つ事に注目します。$n < 2^{M/2}$ となるように M を選べば、$r < n < 2^{M/2}$ となり条件が満たされます。$n < 2^{M/2}$ を整理すると $M \geq 2\lceil \log_2 n \rceil$ という条件が導出できます。

では M を大きくすれば良いのでは？

理論上は M を大きくするほど精度が上がるのですが、実用上は以下の制約があります。

- **量子ビット数の増加**：M 個の制御ビットが必要
- **回路深さの増加**：制御モジュラー累乗の回路が深くなる
- **ノイズの蓄積**：回路が深いほど誤りが蓄積し、測定結果が信頼できなくなる

現在の NISQ（Noisy Intermediate-Scale Quantum）デバイスではノイズが大きな問題であり、M を必要以上に大きくすると逆効果になってしまいます。

コラム：約分の問題

$\gcd(s, r) > 1$ の場合、$\frac{s}{r}$ は既約分数ではないため、収束分数には約分された形で現れます。例えば $s = 2, r = 6$ なら、収束分数は $\frac{1}{3}$ として現れ、分母は $r = 6$ ではなく 3 になります。
この場合、得られた分母は r の約数になります。手順としては、$a^{r'} \equiv 1 \pmod{n}$ を満たさなければ、r' の倍数（$2r', 3r', \ldots$）を試すか、測定からやり直すことになります。ただし、位数は $r < n$ を満たすので、上限として $r' \leq n$ まで試し、ヒットしなければ別の z を測定するか、別の a を選び直します。

連分数展開で位数を求める具体例①（$n = 21$, $a = 2$, $r = 6$, $M = 10$）

$s = 1$ のピークに対応する測定値として $z = 171$ が得られたとする。

$$\frac{z}{2^M} = \frac{171}{1024} = 0.166992...$$

これを連分数展開すると $[0; 5, 1, ...]$ となり、以下の収束分数が挙げられます

打ち切り位置	収束分数	値
$[0]$	$\frac{0}{1}$	0
$[0; 5]$	$\frac{1}{5}$	0.2

打ち切り位置	収束分数	値
$[0;5,1]$	$\frac{1}{6}$	0.1666... **分母 6 が位数 r と一致する**

分母 $r' = 6$ で検証 $\rightarrow 2^6 \bmod 21 = 64 \bmod 21 = 1$

連分数展開で位数を求める具体例②（$n = 15, a = 7, M = 10$）

測定値 $z = 256$ が得られたとする。

$$\frac{z}{2^M} = \frac{256}{1024} = 0.25$$

連分数展開を行う。

$$0.25 = \frac{1}{4} = 0 + \frac{1}{4} = [0;4]$$

収束分数を計算する。

展開	収束分数	分母
$[0]$	$0/1$	1
$[0;4]$	$1/4$	4

分母 $r' = 4$ を位数の候補として検証する。

$$7^4 \bmod 15 = 2401 \bmod 15 = 1$$

位数 $r = 4$ が得られた。

ステップ 6：位数から素因数を計算

位数 r が求まれば、3.2 節で示した方法で素因数を計算する。

手順

1. r が偶数であることを確認
   - 奇数なら別の a を選んでやり直し
2. $a^{r/2} \not\equiv -1 \pmod{n}$ を確認
   - -1 に等しいなら別の a を選んでやり直し
3. 素因数を計算

$$p = \gcd(a^{r/2} + 1, n), \quad q = \gcd(a^{r/2} - 1, n) \tag{3.25}$$

具体例（$n = 15, a = 7, r = 4$）

チェック 1：$r = 4$ は偶数

チェック 2：$a^{r/2} = 7^2 = 49$

$$49 \bmod 15 = 4 \neq 14 \equiv -1 \pmod{15}$$

素因数の計算は以下の通りである。

$$p = \gcd(49+1, 15) = \gcd(50, 15)$$

ユークリッド互除法を実施すれば、

- $50 = 15 \times 3 + 5$
- $15 = 5 \times 3 + 0$

となり、したがって、$\gcd(50, 15) = 5$ が得られた。

$$q = \gcd(49-1, 15) = \gcd(48, 15)$$

もう一方も、ユークリッド互除法を実施する。

- $48 = 15 \times 3 + 3$
- $15 = 3 \times 5 + 0$

よって $\gcd(48, 15) = 3$ が得られた。

$$\boxed{15 = 3 \times 5}$$

以上より、$n = 15$ の因数分解が完了した。

3.4.3　手順のまとめ

以下では、各ステップで「何を決めるか（意思決定）」と「何が確定するか」を明示する。

入力：n（二つの異なる素数 p, q の合成数）

この時点で確定しているのは n のみ。n の因数 p, q は未知である。

ステップ 0：a を選ぶ（★ a は我々の意思決定で定まる）

操作	$1 < a < n$ の範囲から a をランダムに選び、$\gcd(a, n)$ を計算
結論	量子回路 $U_{a,n}$ が確定する。位数 r も数学的に確定するが、r の値は我々には未知

分岐

- $\gcd(a, n) \neq 1 \to \gcd(a, n)$ がそのまま因数 → **アルゴリズム終了**
- $\gcd(a, n) = 1 \to$ ステップ 1 へ

ステップ 1：初期状態の準備

操作	制御レジスタ $\mathcal{M}$ を $\|0\rangle$、ターゲットレジスタ $\mathcal{N}$ を $\|1\rangle$ に初期化
結論	量子状態 $\|0\rangle_{\mathcal{M}} \otimes \|1\rangle_{\mathcal{N}}$ が準備される

制御ビット数 M は事前に $M \geq 2\lceil \log_2 n \rceil$ を満たすよう設定しておく。

ステップ 2：アダマール変換

操作	制御レジスタ $\mathcal{M}$ の各ビットにアダマールゲート H を適用
結論	2^M 通りの状態が均等に重ね合わさる → 並列計算の準備完了

ステップ 3：制御モジュラー累乗

操作	制御レジスタ $\mathcal{M}$ の値 y に応じて、ターゲット $\mathcal{N}$ に $a^y \bmod n$ を計算
結論	$\mathcal{M}$ と $\mathcal{N}$ がエンタングル → 位数 r の周期構造が状態に埋め込まれる

※ a を選んだ時点で、制御モジュラー乗算に掛ける定数 $a^{2^k} \bmod n$ は古典的に事前計算でき、回路のゲート列を組み立てられる。

ステップ 4：逆 QFT

操作	制御レジスタ $\mathcal{M}$ に逆量子フーリエ変換を適用
結論	周期 r の情報が「測定可能な形」に変換される → $z \approx \frac{2^M s}{r}$ にピーク出現

ステップ 5：測定

操作	制御レジスタ $\mathcal{M}$ を計算基底で測定
結論	測定値 z が得られる → ここから古典計算で r を抽出する

※ どの s に対応するピークが出るかは量子力学的に確率的。我々には関与できない部分。

分岐

- $z = 0$：r の情報が得られない → **再測定**または**ステップ 0 へ**
- $z \neq 0$：→ 連分数展開へ

ステップ 5 続き：連分数展開（古典計算）

操作	$\frac{z}{2^M}$ を連分数展開し、各収束分数の分母 r' に対して $a^{r'} \equiv 1 \pmod{n}$ を検証
結論	真の位数 r を特定（$\gcd(s, r) > 1$ のときは r の約数が得られるので倍数を試す）

検証：各 r' に対して $a^{r'} \equiv 1 \pmod{n}$ を確認

検証結果	次のアクション
成り立つ	収束分数の分母上限 $r' \leq r/\gcd(s, r) \leq r$ と位数の最小性 $r \leq r'$ から $r' = r$。→ ステップ 6 へ
成り立たない	連分数展開の分母は $r/\gcd(s, r)$ なので $\gcd(s, r) > 1$ のときは r の約数が得られている。その倍数（$2r'$, $3r'$, ...）を試す
それでも見つからない	→ **再測定**または**ステップ 0 へ**（別の a を選ぶ）

ステップ 6：因数分解（古典計算）

操作	(1) r が偶数か確認 → (2) $a^{r/2} \not\equiv -1$ か確認 → (3) gcd を計算
結論	因数 p, q を得る、または別の a でやり直し

検証 1：r は偶数か？

- 奇数 → **ステップ 0 へ**（別の a を選ぶ）
- 偶数 → 検証 2 へ

検証 2：$a^{r/2} \bmod n$ の値は？

- $\equiv -1 \pmod{n}$ → **ステップ 0 へ**（別の a を選ぶ）
- $\not\equiv -1 \pmod{n}$ → 因数を計算

因数の計算を実施

$$p = \gcd(a^{r/2} + 1, n), \quad q = \gcd(a^{r/2} - 1, n)$$

→ **因数分解完了**

3.5　実用性の分析

ここまで Shor アルゴリズムの手順を見てきた。本章では、Shor アルゴリズムの実用性を多角的に分析する。

本章で議論するのは以下の 2 点である。

1. **計算量の優位性**
   Shor アルゴリズムは、現状人類が知っている古典アルゴリズムと比較してどれほど高速なのか、それが RSA 暗号の安全性にどう影響するのか（3.5.3, 3.5.4 節）
2. **技術課題**
   現状の技術で実現可能なのか（3.5.6 節）

3.5.1　「解ける・解けない」とスケールの関係

「Shor アルゴリズムなら因数分解が解ける」という説明を聞いて、「小さな数なら古典でも解けるのでは？」と思った読者もいるかもしれない。実際、$n = 15$ のような小さな数なら、古典アルゴリズムでも一瞬で因数分解できる。

ここで重要なのは、**「解ける・解けない」という議論は、問題のスケール（暗号化に使う数の大きさ）を増やしたときの振る舞いについての話だという点である。**

- **古典アルゴリズム（準指数時間）**
  - 小さな数（例：$n = 15$）なら瞬時に解ける
  - しかし、数を大きくしていくと、計算時間が爆発的に増加
  - RSA-2048 のような暗号で使う大きな数では数百万年 → **現実的に解けない**

- **Shor アルゴリズム（多項式時間）**
  - 小さな数でも大きな数でも、スケールに対する増加速度は緩やか
  - RSA-2048 にスケールアップしても多項式時間のオーダーで解ける → **古典計算とは桁違いに高速**

つまり、RSA 暗号は「数を大きくすれば古典計算では解けなくなる」という性質で安全性を担保していた。しかし、Shor アルゴリズムはこの前提を覆す。**どれだけ数を大きくしても、多項式時間で解けてしまう**ため、暗号としての安全性が成立しなくなる。これが、量子コンピュータが現代暗号に対する脅威と言われる根本的な理由である。

計算量の優位性を定量的に評価するためには、アルゴリズムの全体計算量を見積もる必要がある。全体計算量は「1 試行あたりの計算量 × 期待試行回数」で決まる。そこで、まず 3.5.3 節で 1 回の試行の計算量を評価し、次に 3.5.4 節で成功確率と必要な試行回数を分析し、全体の計算量を導出する。

これらを正しく理解するためには、3.4 節で示した理論的な説明と、実際の量子回路実装の間にあるギャップを埋める必要がある。そこで、まず 3.5.2 節では実装方法を説明する。

3.5.2　実装方法

3.4 節では、Shor アルゴリズムの手順を理論的に説明した。本節では、それを実際に量子回路として実装する際のポイントを説明する。

制御モジュラー累乗の実装

3.4.3 節のステップ 3 では、「制御レジスタ $\mathcal{M}$ の値 y に応じて、ターゲット $\mathcal{N}$ に $a^y \bmod n$ を計算」と説明した。

つまり、以下のユニタリ変換を実現する必要がある。

$$U\Big(|y\rangle \otimes |x\rangle\Big) = |y\rangle \otimes |x \cdot a^y \bmod n\rangle$$

制御レジスタが重ね合わせ状態のとき、この変換は以下のように作用する。

$$U\left(\left(\sum_{y=0}^{2^M-1} \alpha_y |y\rangle\right) \otimes |x\rangle\right) = \sum_{y=0}^{2^M-1} \alpha_y |y\rangle \otimes |x \cdot a^y \bmod n\rangle$$

つまり、各 $|y\rangle$ 成分に対して、**異なる値 a^y を乗算する**必要がある。この操作を量子回路でどう実現するかが、本節の主題である。

2 進数展開によるゲート数の削減

制御モジュラー累乗 $|y\rangle \otimes |x\rangle \rightarrow |y\rangle \otimes |x \cdot a^y \bmod n\rangle$ を実装する際、定義通りに各 y の値（$y = 0, 1, \ldots, 2^M - 1$）に対してユニタリ演算 U_y を個別に用意すると、2^M 個のゲートが必要になる。RSA-2048 では $M \geq 4096$ なので、2^{4096} 個となり実装不可能である。

そこで、2 進数展開を利用してこの数を M 個に削減する。

y は M ビットの整数なので、2 進数で表すと以下のように書ける。

$$y = \sum_{k=0}^{M-1} y_k \cdot 2^k$$

ここで $y_k \in \{0,1\}$ は y の k ビット目の値である。このとき、$a^y \bmod n$ は以下のように計算できる。

$$a^y = a^{\sum_{k=0}^{M-1} y_k \cdot 2^k} = \prod_{k=0}^{M-1} (a^{2^k})^{y_k} \pmod n = a^{2^0 \cdot y_0} \cdot a^{2^1 \cdot y_1} \cdot \ldots \cdot a^{2^{M-1} \cdot y_{M-1}}$$

したがって、どんな y も、$a^{2^0}, a^{2^1}, \ldots, a^{2^{M-1}}$ という M 種類の値の積（の組み合わせ）で表現できる。

そして、量子回路では、**この積を M 個の制御ゲートの順次適用**として実現する。

$$|x\rangle \xrightarrow{U_{2^0}^{y_0}} |x \cdot a^{2^0 \cdot y_0}\rangle \xrightarrow{U_{2^1}^{y_1}} |x \cdot a^{2^0 \cdot y_0} \cdot a^{2^1 \cdot y_1}\rangle \xrightarrow{U_{2^2}^{y_2}} \cdots \xrightarrow{U_{2^{M-1}}^{y_{M-1}}} |x \cdot a^y\rangle$$

$U_{2^k}^{y_k}$ は、$y_k = 0$ のとき、$U_{2^k}^0 = I$（恒等演算）となり、$y_k = 1$ のとき、$U_{2^k}^1 = U_{2^k}$（a^{2^k} を掛ける発展）である。つまり、単一の制御ゲートの処理は以下のように書ける。

$$|y_0, \ldots, y_{k-1}\rangle \otimes |y_k\rangle \otimes |y_{k+1}, \ldots, y_{M-1}\rangle_{\mathcal{M}} \otimes |x\rangle_{\mathcal{N}} \to |y_0, \ldots, y_{k-1}\rangle \otimes |y_k\rangle \otimes |y_{k+1}, \ldots, y_{M-1}\rangle_{\mathcal{M}} \otimes U_{2^k}^{y_k} |x\rangle_{\mathcal{N}}$$

重要なのは、各制御ゲートは、**1 個の制御ビット**（y_k）のみを持ち、他のビット y_j（$j \neq k$）は無関係という点である。

この M 個の制御ゲートで構成される一つの回路で、すべての y（2^M 通り）に対応することができる。

古典前計算の必要性

制御ゲート U_{2^k} は「$a^{2^k} \bmod n$ を掛ける」という演算を行うが、これを量子回路として実装するには、$a^{2^k} \bmod n$ の具体的な数値が必要である。この数値を使って、量子ゲートの具体的な回路のパラメータを調節する。

したがって、上記の説明では話の流れを優先して飛ばしたが、量子回路を実行する前に、古典コンピュータで $a^{2^k} \bmod n$（$k = 0, \ldots, M-1$）を事前計算する必要がある。

事前計算は以下のように、繰り返し二乗法で行う。各ステップで前のステップの結果を二乗するだけなので、M 回の乗算で計算できる。

- $a^{2^0} = a$
- $a^{2^1} = (a^{2^0})^2 \bmod n$
- $a^{2^2} = (a^{2^1})^2 \bmod n$
- $\vdots$
- $a^{2^{M-1}} = (a^{2^{M-2}})^2 \bmod n$

Shor のアルゴリズムでは、失敗した場合に新しい a を選んで再試行する。その際、新しい a に対して再度この古典前計算を行い、量子回路のパラメータを変更する必要がある。ただし、回路の構造（ゲートの種類や接続）は a によらず同じであり、変わるのは各ゲートのパラメータ（定数 $a^{2^k} \bmod n$ の値）のみである。

この事前計算により、量子回路では各「制御ゲート U^{2^k}」を定数 $a^{2^k} \bmod n$ を使った固定回路として実装することができる。

コラム：パラメータ変更の物理的実装

「量子回路のパラメータを変更する」とは、物理的には何を意味するのかいまいちピンと来ないかと思います。ここでは、超伝導量子ビットを例に説明します。

超伝導量子ビットでは、ゲート操作はマイクロ波パルスを照射することで実現します。例えば、回転ゲート $R_z(\theta)$ は、特定の周波数のマイクロ波パルスを一定時間照射することで実装します。パラメータ θ の値を変えるには、パルスの長さや位相を変えれば良いです。

モジュラー乗算ゲートも、基本ゲート（CNOT や回転ゲートなど）の組み合わせで実装され、a が変わっても、回路の構造（ゲートの種類や順序）は同じであり、制御 PC から各ゲートのパラメータ（パルスの長さや位相など）を更新するだけで済みます。回路を再設計する必要はありません。

具体例（$y = 5 = 0101_2$ の場合）

本回路では、ビット数 $M = 4$ なので、4 つの制御ゲートで構成される。なお、本書では計算基底のケット表記を $|y_0, y_1, \ldots, y_{M-1}\rangle$ の順（最下位ビット先頭）で書く。$y = 5 = 0101_2$ は $y_0 = 1, y_1 = 0, y_2 = 1, y_3 = 0$ なので、ケット表記では $|1, 0, 1, 0\rangle$ となる。

⓪ 初期状態

$$|y\rangle_{\mathcal{M}} \otimes |x\rangle_{\mathcal{N}} = |1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x\rangle_{\mathcal{N}}$$

① $k = 0$ のステップ

制御ビット $y_0 = 1$ に基づいて制御ゲート $U_{2^0}^{y_0}$ を適用する。

$$|1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x\rangle_{\mathcal{N}} \xrightarrow{U_{2^0}^{y_0}} |1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^1\rangle_{\mathcal{N}}$$

② $k = 1$ のステップ

制御ビット $y_1 = 0$ に基づいて制御ゲート $U_{2^1}^{y_1}$ を適用する（$y_1 = 0$ なので変化なし）。

$$|1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^1\rangle_{\mathcal{N}} \xrightarrow{U_{2^1}^{y_1}} |1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^1\rangle_{\mathcal{N}}$$

③ $k = 2$ のステップ

制御ビット $y_2 = 1$ に基づいて制御ゲート $U_{2^2}^{y_2}$ を適用する。

$$|1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^1\rangle_{\mathcal{N}} \xrightarrow{U_{2^2}^{y_2}} |1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^5\rangle_{\mathcal{N}}$$

④ $k = 3$ のステップ

制御ビット $y_3 = 0$ に基づいて制御ゲート $U_{2^3}^{y_3}$ を適用する（$y_3 = 0$ なので変化なし）。

$$|1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^5\rangle_{\mathcal{N}} \xrightarrow{U_{2^3}^{y_3}} |1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^5\rangle_{\mathcal{N}}$$

⑤ 最終状態

$$|1, 0, 1, 0\rangle_{\mathcal{M}} \otimes |x \cdot a^5\rangle_{\mathcal{N}}$$

この例では $M = 4$ なので、4 つの制御ゲート $U_{2^0}^{y_0}, U_{2^1}^{y_1}, U_{2^2}^{y_2}, U_{2^3}^{y_3}$ で構成されていた。一方で、実際の RSA-2048 を解くには $M \geq 4096$ が必要なので、この回路には 4096 個以上の制御ゲートが並ぶことになる。このように、スケールが大きくなっても必要なゲート数は M 個（2^M ではない）であり、これが Shor のアルゴリズムが効率的に動作する理由の一つである。

逆量子フーリエ変換の実装

3.4.3 節のステップ 4 では、制御レジスタに逆量子フーリエ変換（IQFT）を適用する。ここでは、QFT 回路の構成を説明する。

QFT 回路は、アダマールゲートと制御位相ゲートで構成される。M qubit の QFT 回路の構成は以下の通りである。

1. **アダマールゲート**
   各 qubit に 1 つずつ適用する。合計 M 個。
2. **制御位相ゲート**
   qubit i（$i = 1, \ldots, M-1$）に対して、qubit $i+1, i+2, \ldots, M$ を制御ビットとする位相ゲートを適用する。qubit i には $M - i$ 個の制御位相ゲートが作用するため、総数は以下のように計算できる。
   $$(M-1) + (M-2) + \cdots + 1 = \frac{M(M-1)}{2}$$

コラム：計算量の漸近記法（O, Ω）

本節では計算量を評価するために、O と Ω の漸近記法を使用します。

O 記法（上界）

- $f(n) = O(g(n))$ は、「f の増加速度は g を超えない」という意味です
- 例：$3n^2 + 5n = O(n^2)$
  - n が大きくなると、$3n^2$ の項が支配的になります
  - 定数倍（3）や低次項（$5n$）は無視できます
  - 結果として n^2 に比例する程度で増加します

Ω 記法（下界）

- $f(n) = \Omega(g(n))$ は、「f の増加速度は少なくとも g 程度ある」という意味です
- 例：成功確率が $\Omega(1/\log\log n)$ なら、少なくとも $1/\log\log n$ 程度の確率で成功します
- 性能の保証（下限）を示すときに使います

3.5.3　Shor のアルゴリズムの計算量（1 回の試行）

本節では、素数が見つかる成功パターンに沿って、Shor のアルゴリズムの 1 回の試行の計算量を詳細に評価する。

量子アルゴリズムには量子計算と古典計算の両方が含まれる。アルゴリズムが「多項式時間」であることを示すには、量子部分（量子ゲートの数）と古典部分（ビット演算数）の両方が n の多項式であることを確認すればよい。本節では、Shor アルゴリズムの量子部分と古典部分がともに $O((\log n)^3)$

であることを確認する。

パラメータ設定

制御レジスタのビット数 M は、測定精度を保証するために $M \geq 2\lceil \log_2 n \rceil$ と設定する（3.4 節で説明した通り）。したがって、以下のとおりである。

$$M = O(\log n)$$

つまり、M は n の対数オーダーである。以下の計算量解析ではこの関係を用いる。

(1) 古典前計算

3.5.2 節で説明したように、制御モジュラー累乗を実装するには、量子回路を実行する前に、古典コンピュータで $a^{2^k} \bmod n$（$k = 0, 1, \ldots, M-1$）を繰り返し二乗法で事前計算する必要がある。この計算量は以下のように評価できる。

繰り返し二乗法により、$a^{2^0}, a^{2^1}, \ldots, a^{2^{M-1}}$ の M 個の値を順次計算していく。各ステップでは前のステップの結果を二乗するため、合計 M 回の乗算が必要となる。n は $\log_2 n$ ビットの整数なので、各乗算（$\log_2 n$ ビットの整数同士の乗算と mod 演算）には $O((\log n)^2)$ ビット演算を要する（例：RSA-2048 では $n \approx 2^{2048}$ なので、2048 ビットの整数同士の乗算となる）。したがって、M 回の乗算の総計算量は

$$O(M \cdot (\log n)^2) = O(\log n \cdot (\log n)^2) = O((\log n)^3)$$

となる。

※ この部分は量子ゲート数には含まれないが、全体の実行時間には影響する。

(2) 制御モジュラー乗算

3.5.2 節で説明した実装方法に従い、制御レジスタ $\mathcal{M}$ の各ビット k（$k = 0, \ldots, M-1$）に対して、制御ゲート U_{2^k} を適用する。この計算量は以下のように評価できる。

各制御ゲート U_{2^k} の計算量

U_{2^k} は、ターゲットレジスタ $\mathcal{N}$ の状態 $|x\rangle$ を $|a^{2^k} x \bmod n\rangle$ に写す演算である。

$a^{2^k} \bmod n$ は古典前計算で得られた定数なので、これを c_k と書くと、U_{2^k} は「c_k 倍 mod n」を計算する量子回路である。

モジュラー乗算の量子回路

ターゲットレジスタは $\log_2 n$ ビットである。$\log_2 n$ ビットの整数同士の乗算は、量子加算器を繰り返し適用することで実現される [3]（詳細はコラム参照）。量子加算器 1 回は $O(\log n)$ 個の量子ゲートで構成され、$\log n$ ビット分の加算を $O(\log n)$ 回行うため、乗算全体では $O((\log n)^2)$ 個の量子ゲートとなる。

したがって、各制御ゲート U_{2^k} は $O((\log n)^2)$ 個の量子ゲートで構成される。制御レジスタは $M = O(\log n)$ ビットなので、M 個の制御ゲート全体の計算量は

$$M \times O((\log n)^2) = O(\log n) \times O((\log n)^2) = O((\log n)^3)$$

となり、制御モジュラー累乗全体で $O((\log n)^3)$ 個の量子ゲートが必要となる。

コラム：量子加算器の仕組み

本文では「量子加算器 1 回：$O(\log n)$ 個の量子ゲート」と述べました。ここではリップルキャリー型加算器（Cuccaro et al., 2004）の考え方を簡単に説明します。

古典加算器の量子版

リップルキャリー型加算器は、古典的な筆算の加算を量子回路で再現する方式です。ℓ ビット整数 x と y の加算 $|x\rangle|y\rangle \to |x\rangle|x+y\rangle$ を、下位ビットから 1 ビットずつ繰り上がり（carry）を伝播させて計算します。

ゲート数の評価

各ビット位置での処理は、Toffoli ゲートとパウリゲートなど $O(1)$ 個のゲートで構成されます。ℓ ビットの加算では、これを ℓ 回繰り返すため、合計 $O(\ell)$ 個の量子ゲートとなります。Cuccaro et al. の方式では補助量子ビットも 1 個で済むため、空間効率にも優れています。

Shor アルゴリズムでは $\ell = \lceil \log_2 n \rceil$ なので、量子加算器 1 回あたり $O(\log n)$ ゲートとなり、本文の計算量評価と整合します。

なお、付属ハンズオンで採用する Draper 型（QFT ベースのフーリエ空間算術）でも、QFT を乗算全体の外側にまとめれば加算部分自体は同じ $O(\log n)$ オーダーで済み、Shor 全体の計算量クラス $O((\log n)^3)$ は加算器の方式に依らず同じになります。

(3) 逆量子フーリエ変換（IQFT）

3.4.3 節のステップ 4 で説明した逆量子フーリエ変換の計算量を見積もる。制御レジスタ $\mathcal{M}$（M qubit）に逆 QFT を適用する。

3.5.2 節で説明した QFT 回路の構成より、アダマールゲート M 個と制御位相ゲート $\frac{M(M-1)}{2}$ 個で構成されるため、総ゲート数は以下の通りである。

$$M + \frac{M(M-1)}{2} = \frac{M^2+M}{2} = O(M^2)$$

$M = O(\log n)$ を代入すると、$O((\log n)^2)$ 個の量子ゲートとなることが分かる。

1 試行あたりの総計算量

量子ゲート数

量子回路部分（制御モジュラー乗算 + 逆 QFT）の計算量は以下のとおりである。

$$O((\log n)^3) + O((\log n)^2) = O((\log n)^3)$$

古典計算量

古典前計算（$a^{2^k} \bmod n$ の事前計算）は $O((\log n)^3)$ ビット演算である。

古典部分は量子部分と同等のオーダーであり、全体の計算量を支配しない。したがって、1 試行あたりの計算量は $O((\log n)^3)$ と評価できる。

3.5.4　成功確率と全体の計算量

3.5.3 節では、1 回の試行の計算量が $O((\log n)^3)$ と多項式時間であることを示した。しかし、Shor のアルゴリズムは確率的アルゴリズムであり、1 回の試行で必ず成功するとは限らない。

本節では、成功確率を評価し、必要な試行回数を見積もることで、アルゴリズム全体の計算量を導出する。Shor アルゴリズムがスケールアップしても現実的な時間で解けることを示し、人類に既知の古典アルゴリズムと比較することで現状の優位性を議論する。

失敗の分類：アルゴリズム由来 vs ノイズ由来

Shor のアルゴリズムが失敗する原因は 2 種類に分けられる。一つはアルゴリズム自体の確率的性質に起因するもの、もう一つは量子コンピュータのノイズに起因するものである。本節ではアルゴリズム由来の失敗を分析し、ノイズ由来の失敗については 3.5.6 節で議論する。

分類	原因	対処	本節で扱う
アルゴリズム由来	乱択アルゴリズムとしての確率的性質	試行を繰り返す	本節で扱う
ノイズ由来	量子ハードウェアの誤り	誤り訂正、デバイス改善	3.5.6 節で後述

アルゴリズム由来の失敗は**理想的な量子コンピュータでも発生**するが、試行回数を増やせば解決でき、全体の計算量には多項式オーダーの影響しか与えない。

アルゴリズム由来の失敗①：a の選び方に起因する失敗

量子力学とは無関係で、位数 r の性質に起因する。

失敗条件	発生箇所	確率
r が奇数	3.2.2 節 ステップ 1	$\leq 1/2$
$a^{r/2} \equiv -1 \pmod{n}$	3.2.2 節 ステップ 2	$\leq 1/2$

対処：a を変えてやり直す。両方を回避できる確率は $\geq 1/2$ である（下記コラム「a の選び方と成功確率」参照）。

アルゴリズム由来の失敗②：量子測定の確率的性質に起因する失敗

量子測定は確率的であり、以下の 3 つの要因で失敗しうる。

条件	結果	対処
$s = 0$	ピーク位置が $z = 0$ となり、r の情報が得られない	再試行

条件	結果	対処
QPE の有限精度	最近傍でない z が得られ、連分数展開が正しい s/r を復元できない	再試行
$\gcd(s,r) \neq 1$	連分数展開で s/r が約分され、r の約数 r' しか得られない	再試行

$s = 0$ について

r 個の固有値のうち $s = 0$ に対応するものが選ばれた場合、ピーク位置が $z = 0$ となり位数の情報が得られない。確率は $1/r$ であり、r が大きければ無視できるほど小さい。

QPE の有限精度について

制御レジスタのビット数 M が有限であるため、QPE の測定結果 z は必ずしも $s \cdot 2^M/r$ に最も近い整数とは限らない。3.3 節のステップ 4 で導いたように、最近傍の整数 z_0 が測定される確率は $\geq 4/\pi^2 \approx 0.405$ であり、$M \geq 2\lceil \log_2 n \rceil$ に設定することで、この z_0 から連分数アルゴリズムにより s/r を復元できる（$r < n$ かつ $\gcd(s,r) = 1$ の場合）[6]。

$\gcd(s,r) \neq 1$ について

$\gcd(s,r) = d > 1$ の場合、連分数展開で得られるのは約分後の s/d と $r/d = r'$ であり、真の位数 r ではなくその約数 r' しか得られない。r' の倍数を古典的に探索して r を回復する方法もあるが（3.4 節のコラム「約分の問題」参照）、d が大きい場合は探索範囲が広くなり、多項式時間の保証が難しくなる。そのため、標準的には $\gcd(s,r) = 1$ となる s が得られるまで QPE を再実行する。$\gcd(s,r) = 1$ となる確率は $\phi(r)/r$ であり、Mertens の定理により最悪でも $\Omega(1/\log\log n)$ 程度に留まる。

コラム：a の選び方と成功確率

ランダムに選んだ a に対して、r が偶数かつ $a^{r/2} \not\equiv -1 \pmod{n}$ となる確率が $\geq 1/2$ であることを説明します。

中国剰余定理による分解

$n = pq$（p, q は異なる奇素数）のとき、中国剰余定理により

$$\mathbb{Z}_n^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^*$$

という同型が成り立ちます。つまり、$a \in \mathbb{Z}_n^*$ は $(a \bmod p, a \bmod q)$ の組として表せます。

位数の関係

a の位数を r、$a \bmod p$ の位数を r_p、$a \bmod q$ の位数を r_q とすると、

$$r = \mathrm{lcm}(r_p, r_q)$$

が成り立ちます。

失敗条件の解析

$r_p = 2^{e_p} m_p$、$r_q = 2^{e_q} m_q$（m_p, m_q は奇数）と書くと、以下が成り立ちます。

- r が奇数 $\iff e_p = 0$ かつ $e_q = 0$
- r が偶数かつ $a^{r/2} \equiv -1 \pmod{n} \iff e_p = e_q \geq 1$

つまり、**失敗するのは $e_p = e_q$ のときのみ**です。

確率の評価

$\mathbb{Z}_p^*$ は位数 $p-1$ の巡回群です。$p-1 = 2^{k_p} s_p$（s_p は奇数）と書くと、ランダムな $a \bmod p$ に対して $e_p = k_p$（最大値）となる確率はちょうど $1/2$ で、$e_p < k_p$ となる確率も $1/2$ です。同様に e_q についても成り立ちます。

e_p と e_q は独立に分布するので、$e_p \neq e_q$ となる確率は少なくとも：

- $e_p = k_p$ かつ $e_q < k_q$：確率 $1/2 \times 1/2 = 1/4$
- $e_p < k_p$ かつ $e_q = k_q$：確率 $1/2 \times 1/2 = 1/4$

の和で $\geq 1/2$ となります。したがって、成功確率は $\geq 1/2$ です。

期待試行回数と全体の計算量

Shor アルゴリズムは確率的アルゴリズムなので、成功するまでに複数回の試行が必要な場合がある。ここで「1 試行」とは、a をランダムに選び、量子回路を 1 回実行し、古典的な後処理を行う一連の流れを指す。

成功確率の評価

上記の失敗分析から、1 回の試行で成功するには以下のすべてが成立する必要がある。

1. a の選び方が適切（r が偶数、かつ $a^{r/2} \not\equiv -1$）：確率 $\geq 1/2$
2. $s \neq 0$（位相の情報が得られる）：確率 $1 - 1/r$
3. QPE が最近傍の z を出力（連分数で復元可能）：確率 $\geq 4/\pi^2$
4. $\gcd(s, r) = 1$（連分数で真の位数 r が復元される）：確率 $\phi(r)/r$

したがって、1 回の試行で成功する確率は以下のとおりである。

$$p \geq \frac{1}{2} \times \left(1 - \frac{1}{r}\right) \times \frac{4}{\pi^2} \times \frac{\phi(r)}{r}$$

（注: 厳密には $(1-1/r)$ と $\phi(r)/r$ は独立でない。$\gcd(0, r) = r \neq 1$ なので、より簡潔には Nielsen & Chuang [2] の形で $p \geq (4/\pi^2) \cdot \phi(r)/r$ と書ける。いずれにせよ漸近結論 $p = \Omega(1/\log\log n)$ は変わらない。）

Mertens の定理（数論の標準的事実: $\prod_{p \leq x}(1 - 1/p)$ は調和級数の逆数オーダー $1/\log x$ で減衰する）を用いる。より具体的には、r が小さい素数の積で構成される最悪ケースでも、素数定理から最大の素因数 $p_{\max} \sim \log r$ 程度なので、$\phi(r)/r = \prod_{p|r}(1 - 1/p) \geq \prod_{p \leq \log r}(1 - 1/p) = \Omega(1/\log\log r)$ となる（log が 1 段深まる）。したがって $\phi(r)/r$ は最悪でも $\Omega(1/\log\log r)$ であり、$r < n$ なので、$p = \Omega(1/\log\log n)$ が成り立つ。

期待試行回数

試行回数を T とする。各試行は独立で成功確率 p なので、k 回目で初めて成功する確率 $P(T=k)$ は以下のように書ける。

$$P(T=k) = (1-p)^{k-1} \cdot p$$

成功するまでの試行回数の期待値は以下のように計算でき、

$$E[T] = \sum_{k=1}^{\infty} k \cdot P(T=k) = \sum_{k=1}^{\infty} k \cdot (1-p)^{k-1} \cdot p = \frac{1}{p}$$

$p = \Omega(1/\log\log n)$ なので、期待試行回数は $E[T] = O(\log\log n)$ となる。これは入力サイズ $L = \log n$ の多項式よりも緩やかに増加する量であり、全体の多項式時間性に影響を与えない。

注記：$O(\log\log n)$ は理論的な最悪ケースの保証である。最悪ケースとは r が非常に多くの小さな素因数を持ち、$\phi(r)/r$ が極端に小さくなる場合であり、ランダムに選んだ a に対してこのような r に遭遇する確率は極めて低い。実用上は、ほとんどの場合 $O(1)$ 回の試行で成功する。

コラム：試行回数の期待値の計算

$q = 1-p$ とおきます。幾何級数の公式より、$|q| < 1$ のとき

$$\sum_{k=0}^{\infty} q^k = \frac{1}{1-q} = \frac{1}{p}$$

が成り立ちます。両辺を q で微分すると

$$\sum_{k=1}^{\infty} k \cdot q^{k-1} = \frac{1}{(1-q)^2} = \frac{1}{p^2}$$

となります。したがって以下の関係が得られます。

$$\sum_{k=1}^{\infty} k \cdot (1-p)^{k-1} \cdot p = p \cdot \frac{1}{p^2} = \frac{1}{p}$$

全体の計算量

3.5.3 節の冒頭で述べたように、量子アルゴリズムが「多項式時間」であることを示すには、量子部分と古典部分の両方が n の多項式であることを確認すればよい。

1 試行あたりの計算量（3.5.3 節で評価）

- 量子部分（量子ゲート数）：$O((\log n)^3)$
- 古典部分（事前計算のビット演算数）：$O((\log n)^3)$

期待試行回数（本節で評価）：$O(\log\log n)$（理論的最悪ケース、実用上は $O(1)$）

したがって、全体の計算量は以下のようになる。

$$\boxed{\text{量子ゲート数} = O((\log n)^3 \log\log n), \quad \text{古典ビット演算数} = O((\log n)^3 \log\log n)}$$

n は $L = \log_2 n$ ビットの整数なので、$\log n = O(L)$ である。したがって、

$$O((\log n)^3 \log\log n) = O(L^3 \log L)$$

量子部分・古典部分ともに入力サイズ L の多項式であり、Shor アルゴリズムは**多項式時間**である。$\log L$ の因子は L^3 と比べて緩やかに増加するため、全体の計算量は実質的に $O(L^3)$ と同程度である。**以降、本書では本質を強調する場面では $O(L^3)$ と略記し、厳密な評価が必要な場面では $O(L^3 \log L)$ と書き分ける。**

3.5.5　古典アルゴリズムとの比較

Shor アルゴリズムが入力サイズ L（n のビット数、$L = \log_2 n$）に対して多項式時間 $O(L^3)$ であることを示した。本節では、これが現状人類が明らかにしている古典アルゴリズムと比べてどれほど優位なのかを整理する。

因数分解は古典計算においても長年研究されており、様々なアルゴリズムが開発されてきた。主要な古典アルゴリズムの計算量は以下の通りである。

アルゴリズム	計算量	備考
試し割り	$O(\sqrt{n})$	最も単純だが遅い
Pollard-ρ 法	$O(n^{1/4})$	確率的アルゴリズム
一般数体ふるい法	$O\left(\exp\left(c \cdot (\ln n)^{1/3} (\ln\ln n)^{2/3}\right)\right)$	現在最速（準指数時間）

（以下、自然対数 ln を用いて表記する。$L = \log_2 n$ 表記との換算は $\ln n = (\ln 2)\, L$。）

一般数体ふるい法は「準指数時間」と呼ばれ、純粋な指数時間 $O(2^L)$ よりは速いが、多項式時間ではない。これが「因数分解は計算量的に難しい」と言われる理由である。

古典アルゴリズムでは L が増えると計算時間が爆発的に増加し、RSA-2048（$L = 2048$）程度でも現在のスーパーコンピュータでは天文学的な時間がかかる（下記コラム参照）。一方、Shor アルゴリズムでは計算時間が L^3 に比例して緩やかに増加する。例えば Gidney & Ekerå (2021) [4] は、約 2,000 万個の物理量子ビットを持つ誤り耐性量子コンピュータを仮定した場合、RSA-2048 を約 8 時間で因数分解できると見積もっている。必要なハードウェア規模は現状から桁のギャップがあるが（3.5.6 節で後述）、重要なのは L がスケールしても計算時間が多項式的にしか増えないという点である。これが多項式時間アルゴリズムの強みであり、Shor アルゴリズムが革命的と言われる理由である。

コラム：RSA-2048 を古典で解くにはどれくらいかかるのか

実績から見る困難さ

2020 年に RSA-250（829 ビット）の因数分解が達成されましたが、約 2700 CPU コア年を要しました。これは 2026 年 4 月時点での一般目的アルゴリズムによる公開最大記録です。RSA-2048（2048 ビット）はこれよりも桁違いに困難であり、現在のスーパーコンピュータでも現実的な時間では解けません。

漸近式によるオーダー感

一般数体ふるい法の計算量は $L_n[1/3, c] = \exp\left(c \cdot (\ln n)^{1/3} (\ln \ln n)^{2/3}\right)$ で与えられます。$n = 2^{2048}$, $c \approx 1.923$ を代入すると、$\ln n \approx 1420$ より、

$$\exp(1.923 \times 1420^{1/3} \times (\ln 1420)^{2/3}) \approx \exp(81) \approx 10^{35}$$

オーダーの演算が必要になります。これは 2^{117} 程度に相当し、後述する NIST の「112 ビットセキュリティ」評価とも整合します。定数 c の取り方や演算の単位によって結果は変動するため、この数値は正確な予測ではなく、直観的なオーダー感を与えるものです。いずれにせよ、現代のスーパーコンピュータの演算能力（約 10^{18} FLOPS）と比較しても現実的な時間では到達できない値であり、実行不可能であることが分かります。

公的な安全性評価

NIST（米国標準技術研究所）は SP 800-57 において RSA-2048 を「112 ビットセキュリティ」と評価しており、古典計算による解読は実行不可能とされています。

3.5.6　現状の技術での課題

理想と現実

3.5.4 節まで、Shor アルゴリズムが理論的には、$L = \log_2 n$ となる n のビット長を用いて、多項式時間 $O(L^3)$ で因数分解を解けることを示した。しかし、この議論は**理想的な量子ビット**を前提としていた。

現実の量子ビットは様々なノイズの影響を受け、エラーが発生する。各ゲートには一定のエラー率があり、ゲート数が増えるほど正しい結果を得る確率が下がる。

この問題を解決するのが**誤り訂正**である。しかし、量子の誤り訂正は古典とは根本的に異なり、非常に難しい。

古典ビットの誤り訂正

当然、古典コンピュータでもエラーは発生し、誤り訂正は行われている。古典ビットは 0 か 1 の離散値なので、同じビットを複数コピーして多数決を取れば誤りを訂正できる。これは技術的に容易であり、現代のコンピュータでは誤り訂正が十分に機能している。

量子ビットの誤り訂正が難しい理由

一方で以下の理由から量子ビットの誤り訂正は技術的なハードルが高い。

- **No-Cloning Theorem**
  量子状態はコピーできない。古典のように「同じ状態を複数コピーして多数決」という方法がとれない。
- **連続的な誤り**
  古典ビットは $0 \to 1$ のビット反転だけだが、量子ビットは位相の連続的なずれも誤りとなる。
- **測定による破壊**
  誤りを検出しようと測定すると、量子状態自体が壊れる。

これらの制約を克服するため、量子誤り訂正では複数の物理量子ビットを使って誤り訂正を行い、あたかもエラーのない 1 つの量子ビットのように振る舞う「**論理量子ビット**」を構成する。3.5.4 節

までの議論は、暗黙のうちにこの論理量子ビットを前提としていた。

コラム：量子誤り訂正が難しい 3 つの理由

本文で挙げた 3 つの困難を、より詳しく説明します。

1. No-Cloning Theorem

「任意の未知の量子状態をコピーする操作は存在しない」という定理です。

量子状態 $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ を 3 つコピーして $|\psi\rangle|\psi\rangle|\psi\rangle$ を作ることは、物理法則により不可能です。

証明（背理法）任意の状態をコピーできるユニタリ演算子 U が存在すると仮定します。

$$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle, \quad U|\phi\rangle|0\rangle = |\phi\rangle|\phi\rangle$$

両辺の内積を取ると以下のように計算することができます。

$$\langle\psi|\langle 0|U^\dagger U|\phi\rangle|0\rangle = \langle\psi|\psi\rangle\langle\psi|\phi\rangle\langle\phi|\phi\rangle$$

U はユニタリなので $U^\dagger U = I$ より以下が成り立ちます。

$$\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2$$

これを満たすのは $\langle\psi|\phi\rangle = 0$（直交）または $\langle\psi|\phi\rangle = 1$（同一）の場合のみです。つまり、直交でも同一でもない 2 つの状態を両方コピーできる U は存在しません。

2. 連続的な誤り（位相誤り）

古典ビットの誤りは「$0 \to 1$」か「$1 \to 0$」の 2 種類だけです。量子ビットの誤りを理解するには、**Bloch 球**を使うと分かりやすくなります。

Bloch 球による量子ビットの表現

任意の 1 量子ビット状態は、単位球面上の点として表せます：

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle$$

- θ：北極からの角度（$0 \le \theta \le \pi$）
- ϕ：赤道面での角度（$0 \le \phi < 2\pi$）。$|0\rangle$ と $|1\rangle$ の相対位相に対応します

状態	Bloch 球上の位置	(θ, ϕ)
$\|0\rangle$	北極	$(0, -)$
$\|1\rangle$	南極	$(\pi, -)$
$\|+\rangle = \frac{1}{\sqrt{2}}(\|0\rangle + \|1\rangle)$	赤道（+X 方向）	$(\pi/2, 0)$
$\|-\rangle = \frac{1}{\sqrt{2}}(\|0\rangle - \|1\rangle)$	赤道（− X 方向）	$(\pi/2, \pi)$

Bloch 球で見る誤り

誤りの種類	Bloch 球での操作	効果
ビット反転（X 誤り）	X 軸周りに 180° 回転	$\|0\rangle \leftrightarrow \|1\rangle$（極が入れ替わる）
位相反転（Z 誤り）	Z 軸周りに 180° 回転	$\|+\rangle \leftrightarrow \|-\rangle$（赤道上で反対側へ）
連続的な位相誤り	Z 軸周りに δ 回転	$e^{i\phi} \to e^{i(\phi+\delta)}$（赤道上を少しずれる）

古典では「北極か南極か」の 2 択なので、誤りは「極の入れ替え」だけです。しかし量子では赤道上の位置（位相 ϕ）も情報を持つため、Z 軸周りの回転も誤りになります。しかも、この回転角度 δ は連続的な値を取りえます。
幸い、量子誤り訂正の理論では、任意の誤りはパウリ行列 X, Y, Z の線形結合で表せることが示されています。そのため、離散的なパウリ誤りを訂正できれば、連続的な誤りも訂正できます。

3. 測定による状態の破壊と間接測定

古典ビットは、読み取っても値が変わりません。しかし量子ビットに対する射影測定では、状態が測定基底に射影され、重ね合わせが壊れます。

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \xrightarrow{\text{射影測定}} |0\rangle \text{ または } |1\rangle$$

これでは「誤りがあるかチェックする」こと自体が、計算中の量子状態を破壊してしまいます。
量子誤り訂正では、この問題を複合系の間接測定で解決します。エラーがあるか確認したい qubit を直接測定する代わりに、補助 qubit を介して誤りの情報だけを取り出すという考えです。

```
データ qubit  |ψ> ──────●────────  |ψ> のまま
                       |
補助 qubit    |0> ──────⊕────────  測定 → 誤りの情報
```

1. 補助 qubit をデータ qubit とエンタングルさせる（CNOT など）
2. 補助 qubit だけを測定する
3. 測定結果から「どの誤りが起きたか」が分かる
4. データ qubit は測定していないので、重ね合わせは保たれる

この「誤りの種類」を検出する測定を**シンドローム測定**と呼びます。
量子誤り訂正符号（表面符号など）では、シンドロームから「どの qubit にどのパウリ誤り（X, Y, Z）が起きたか」が特定できるよう設計されています。パウリ演算子は $X^2 = Y^2 = Z^2 = I$ を満たすので、同じ演算子をもう一度適用すればエラーが修正できます。
量子誤り訂正の詳細については Nielsen & Chuang [2] の第 10 章を参照してください。

NISQ デバイスと Shor のアルゴリズム

現在主流の量子コンピュータは **NISQ**（Noisy Intermediate-Scale Quantum：ノイズあり中規模量子）と呼ばれ、誤り訂正機能を持たない。NISQ デバイスでは、「ノイズがあっても意味のある結果が得られるタスク」すなわち、浅い回路で済む変分アルゴリズムなどで効果を発揮している。一方で、Shor アルゴリズムのように回路が深いアルゴリズムでは、エラーが蓄積して正しい結果が得られない。

現在の量子ゲートのエラー率は 1 ゲートあたり約 10^{-3} 程度であるが、Shor アルゴリズム（RSA-2048）では約 10^9 個のゲートが必要なので、誤り訂正なしで全ゲートが成功する確率は $(1-10^{-3})^{10^9}$ で、ほぼ 0 である。

前節で議論した通り、Shor アルゴリズムでは、加算・乗算・剰余演算の量子回路から構成されるモジュラー乗算 $|x\rangle \to |ax \bmod n\rangle$ を実装する必要があり、$n = 15$ 程度の小さい数でも数千個のゲートが必要となる。現在の NISQ デバイスで誤差なく実行できるのは数十〜数百ゲート程度であり、汎用版の Shor アルゴリズムでは $n = 15$ すら正しく因数分解することはできない。

コラム：「Shor で因数分解に成功」というニュースについて

量子コンピュータで Shor アルゴリズムを「実行した」というニュースを見たことがあるかもしれません。しかし、それらの多くは特定の n 専用に回路を大幅に簡略化したコンパイル版（toy 版）です。

コンパイル版では、汎用的なモジュラー算術を実装せず、特定の (a, n) に対して「正しい干渉パターンを生む最小回路」を逆算して設計します。そのため、$n = 15$ 用の回路で $n = 21$ は解けません。これは「量子ハードウェアが回路を実行できることのデモ」であり、「因数分解問題を解く能力の実証」ではありません。

本書やハンズオン教材では汎用版を採用し、**シミュレータ上で**「本物の Shor」の動作原理を理解することを重視しています。

ノイズがある場合の Shor アルゴリズム

ここでは、ノイズがあると理想的な場合と比べて Shor のアルゴリズムの挙動がどうなってしまうのかを見る。

理想的な場合（ノイズなし）

3.4 節で見たように、理想的な量子コンピュータでは測定結果のヒストグラムに明確なピークが現れた。ピークの位置は $z \approx \frac{2^M s}{r}$（$s = 0, 1, ..., r-1$）であり、これらのピークから位数 r を求めることができた。

ノイズがある場合

ノイズを加えると、制御レジスタの測定値 z の分布が崩れる。本来の鋭いピークがぼやけ、位数 r を求めるのに必要な正しい z を得る確率が下がる。

ハンズオン教材では、以下で述べるノイズモデルを用いて、シミュレータ上でノイズを加えた Shor アルゴリズムを実行し、理想的な場合との違いを確認する。

ノイズのモデル化

シミュレーションでは、2 量子ビットゲートの脱分極エラー（depolarizing error）でノイズをモデル化した。

脱分極エラーとは

まず 1 量子ビットの場合で説明する。脱分極エラーとは、ゲート適用後に確率 p でランダムなパウリエラーが発生するモデルである。

$$\mathcal{E}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$$

ここで ρ は量子状態（密度行列）、I, X, Y, Z はパウリ行列（および恒等行列）である。

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

直感的には以下のように整理することができる。

- 確率 $(1-p)$：エラーなし（状態 ρ がそのまま）
- 確率 $p/3$：X エラー（ビット反転）
- 確率 $p/3$：Y エラー（ビット反転＋位相反転）
- 確率 $p/3$：Z エラー（位相反転）

2 量子ビットに作用する脱分極チャネルとして、パウリ演算子のテンソル積 $\{I, X, Y, Z\}^{\otimes 2}$ の 16 個のうち、$I \otimes I$（エラーなし）を除いた 15 個の非自明パウリ演算子を等確率で与えるモデルを採用する。

$$\mathcal{E}(\rho) = (1-p)\rho + \frac{p}{15} \sum_{P \in \{I,X,Y,Z\}^{\otimes 2} \setminus \{I \otimes I\}} P\rho P$$

このモデルの妥当性

実際のデバイスでは複数のエラー源が存在する

エラー源	典型的なエラー率	説明
1 量子ビットゲート	~0.1%	Hadamard、位相ゲートなど
2 量子ビットゲート	**~0.5-1%**	CNOT、CZ など
測定エラー	~1-3%	読み出し時の誤り
デコヒーレンス（T1/T2）	回路時間に依存	量子状態の自然な崩壊

※ 超伝導量子ビットの代表的な値 [10]。デバイスや方式（イオントラップ、中性原子等）により具体的な数値は異なるが、2 量子ビットゲートが支配的である構造は共通している。

2 量子ビットゲートのエラーが支配的である理由は、エラー率が 1 量子ビットゲートの 5〜10 倍であり、かつ、Shor アルゴリズムではモジュラー算術の実装に大量の 2 量子ビットゲート（CNOT

等）を必要とするためである。エラー率 p のゲートを n 回適用すると成功確率は $(1-p)^n$ となるため、エラー率とゲート数の両方が大きい 2 量子ビットゲートが支配的になる。

他のエラー源は第一近似で無視できる。1 量子ビットゲートはエラー率が 10^{-3}（$= 0.1\%$）程度、デコヒーレンス（$T_1/T_2 \sim 50$–$100\,\mu$s）はゲート実行時間（$\sim 50\,$ns）より 3 桁長く、測定エラーは回路の最後に 1 回だけで蓄積しない。

以上の理由から、2 量子ビットゲートの脱分極エラーでノイズをモデル化して話を進める。

ノイズが Shor のアルゴリズムの測定確率に与える影響

3.4 節で、理想的な場合の測定確率を密度行列を用いて導出した。ここでは、脱分極エラーがこの測定確率をどのように変化させるかを見る。

理想的な場合（3.4 節のおさらい）

量子回路の出力状態 $|\psi_3\rangle$ に対し、密度行列は純粋状態として

$$\rho^{(\mathrm{ideal})} = |\psi_3\rangle\langle\psi_3|$$

と書ける。ターゲットビットをトレースアウトした制御レジスタの縮約密度行列 $\rho_{\mathcal{M}}$ から、測定確率は式 (3.23) と同じ形

$$P^{(\mathrm{ideal})}(z) = \langle z|\rho_{\mathcal{M}}^{(\mathrm{ideal})}|z\rangle = |S(z)|^2$$

と計算された。ここで $S(z)$ は式 (3.22) で定義した確率振幅である。

脱分極エラーがある場合

各 2 量子ビットゲート U の適用後に、脱分極チャネル $\mathcal{E}$ が作用すると考える。

$$\rho \xrightarrow{U} U\rho U^\dagger \xrightarrow{\mathcal{E}} (1-p)U\rho U^\dagger + \frac{p}{15}\sum_{P\neq I\otimes I} P(U\rho U^\dagger)P$$

n 個の 2 量子ビットゲートを含む回路全体では、密度行列は

$$\rho^{(\mathrm{noisy})} = (\mathcal{E}\circ\mathcal{U}_n)\circ\cdots\circ(\mathcal{E}\circ\mathcal{U}_1)(\rho_0)$$

のように、ゲートとエラーチャネルの合成で記述される。

コヒーレンスの減衰

脱分極チャネルの重要な性質は、密度行列の非対角成分（コヒーレンス）を減衰させることである。簡単のため 1 量子ビットの脱分極チャネルで具体的に計算する。

（注: 本章のシミュレーション（図 3.1）では 2 量子ビット脱分極チャネルを用いているが、以下の解析は定性的な振る舞いを示すための 1 量子ビット版である。2 量子ビット版では収束係数が $1-16p/15$ となるが、定性的な結論（完全混合状態への収束）は同じである。）

密度行列を以下のように準備する

$$\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix}$$

脱分極チャネル $\mathcal{E}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$ の各項を計算する。

まず、各パウリ演算子による変換は以下の通りである。

$$X\rho X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} \rho_{11} & \rho_{10} \\ \rho_{01} & \rho_{00} \end{pmatrix}$$

$$Y\rho Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix} = \begin{pmatrix} \rho_{11} & -\rho_{10} \\ -\rho_{01} & \rho_{00} \end{pmatrix}$$

$$Z\rho Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} \rho_{00} & -\rho_{01} \\ -\rho_{10} & \rho_{11} \end{pmatrix}$$

これらを足し合わせると以下のように計算できる。

$$X\rho X + Y\rho Y + Z\rho Z = \begin{pmatrix} \rho_{00} + 2\rho_{11} & -\rho_{01} \\ -\rho_{10} & 2\rho_{00} + \rho_{11} \end{pmatrix}$$

初期状態は純粋状態 $|\psi\rangle$ なので、密度行列は $\rho = |\psi\rangle\langle\psi|$ であり、規格化条件 $\langle\psi|\psi\rangle = 1$ から $\mathrm{Tr}(\rho) = \rho_{00} + \rho_{11} = 1$ が成り立つ。これを用いて整理する。

- $(0,0)$ 成分
  $\rho_{00} + 2\rho_{11} = \rho_{00} + 2(1 - \rho_{00}) = 2 - \rho_{00}$
- $(1,1)$ 成分
  $2\rho_{00} + \rho_{11} = 2\rho_{00} + (1 - \rho_{00}) = 1 + \rho_{00}$

一方、$2I - \rho$ は以下のように計算できる。

$$2I - \rho = \begin{pmatrix} 2 - \rho_{00} & -\rho_{01} \\ -\rho_{10} & 2 - \rho_{11} \end{pmatrix} = \begin{pmatrix} 2 - \rho_{00} & -\rho_{01} \\ -\rho_{10} & 1 + \rho_{00} \end{pmatrix}$$

したがって、以下が成り立つ。

$$X\rho X + Y\rho Y + Z\rho Z = 2I - \rho$$

この関係を用いて、脱分極チャネルの出力を計算する。

$$\begin{aligned} \mathcal{E}(\rho) &= (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z) \\ &= (1-p)\rho + \frac{p}{3}(2I - \rho) \\ &= (1-p)\rho + \frac{2p}{3}I - \frac{p}{3}\rho \\ &= \left(1 - p - \frac{p}{3}\right)\rho + \frac{2p}{3}I \\ &= \left(1 - \frac{4p}{3}\right)\rho + \frac{2p}{3}I \end{aligned}$$

これを成分で書くと以下のように整理できる。

$$\mathcal{E}(\rho) = \begin{pmatrix} (1 - \frac{4p}{3})\rho_{00} + \frac{2p}{3} & (1 - \frac{4p}{3})\rho_{01} \\ (1 - \frac{4p}{3})\rho_{10} & (1 - \frac{4p}{3})\rho_{11} + \frac{2p}{3} \end{pmatrix}$$

この行列から、非対角成分（ρ_{01}, ρ_{10}）はチャネル適用ごとに $(1 - \frac{4p}{3})$ 倍されることが分かる。対角成分については、$\rho_{ii} = \frac{1}{2}$ を代入すると $(1 - \frac{4p}{3}) \cdot \frac{1}{2} + \frac{2p}{3} = \frac{1}{2}$ となり、$\frac{1}{2}$ が不動点である。

$0 < p < \frac{3}{4}$ のとき $|1 - \frac{4p}{3}| < 1$ である。非対角成分は n 回適用後に $(1 - \frac{4p}{3})^n \rho_{01}^{(0)} \to 0$ となる。

対角成分については、不動点からのズレを計算する。

$$\rho_{ii}^{(n+1)} - \frac{1}{2} = (1 - \frac{4p}{3})\rho_{ii}^{(n)} + \frac{2p}{3} - \frac{1}{2}$$
$$= (1 - \frac{4p}{3})\rho_{ii}^{(n)} - (1 - \frac{4p}{3})\frac{1}{2} = (1 - \frac{4p}{3})\left(\rho_{ii}^{(n)} - \frac{1}{2}\right)$$

すなわち、不動点からのズレも $(1-\frac{4p}{3})$ 倍される。したがって n 回適用後に $(1-\frac{4p}{3})^n(\rho_{ii}^{(0)}-\frac{1}{2}) \to 0$ となり、対角成分は $\frac{1}{2}$ に収束する。

以上より、密度行列は完全混合状態 $\frac{I}{2}$ に収束する。

これは、純粋度 $\mathrm{Tr}(\rho^2)$ が下がり、純粋状態から混合状態へと変化することを意味する。つまり、量子状態のコヒーレンス（重ね合わせによる干渉効果）が失われる。

測定確率への影響

上で示した 1 量子ビットの計算から、脱分極チャネルを繰り返し適用すると密度行列が完全混合状態 $\frac{I}{2}$ に収束することが分かった。多量子ビット系でも同様に、密度行列は完全混合状態 $\frac{I}{2^M}$ に収束する。

ここで 2 つの極限を考える。3.4 節で導出したように、エラーがない理想的な場合の測定確率は

$$P^{(\mathrm{ideal})}(z) = |S(z)|^2$$

であり、$z = s \cdot \frac{2^M}{r}$ 付近に鋭いピークが現れる。一方、エラーが無限に蓄積して完全混合状態になった場合、測定確率は一様分布 $\frac{1}{2^M}$ になる。

実際の回路ではゲート数が有限なので、密度行列はこの 2 つの極限の間にある。したがって測定確率も両者の中間的な振る舞いをし、ピークが低くなり一様分布に近づく。この中間状態の詳細な振る舞いはハンズオンのシミュレーションで数値的に確認できる。図 3.1 に、$n = 15, a = 7$ の場合のシミュレーション結果を示す。左が理想的な場合、中央・右がノイズを加えた場合である。ノイズが増すにつれてピークがぼやけ、一様分布に近づいていく様子が確認できる。

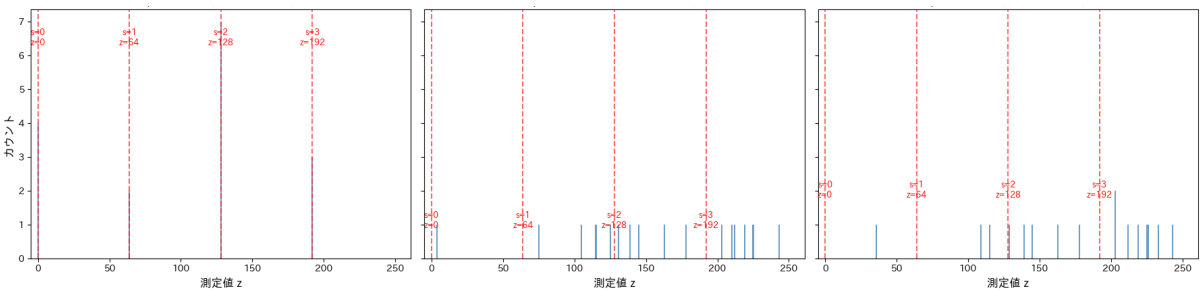

図 3.1　理想的な場合（左）とノイズありの場合（中央・右）の測定値 z のヒストグラム比較。赤い破線は理論的なピーク位置 $z = s \cdot 2^M/r$ を示す。ノイズが増すとピークが崩れ、位数 r の抽出が困難になる。

コラム：T1/T2 緩和と脱分極エラーの違い

本文では脱分極エラー（対称的なノイズモデル）を扱いましたが、実際の量子ビットでは**非対称なデコヒーレンス**が起こります。これを特徴づけるのが T1/T2 緩和時間です。

T1 緩和（縦緩和）
励起状態 $|1\rangle$ がエネルギーを失って基底状態 $|0\rangle$ に遷移する時間スケールです。振幅減衰（amplitude damping）とも呼ばれます。密度行列で見ると、対角成分が $\rho_{11} \to 0$, $\rho_{00} \to 1$ と非対称に変化します。

T2 緩和（横緩和）
位相情報が失われる時間スケールです。純粋位相緩和（pure dephasing）と T1 の影響を合わせたもので、$T_2 \leq 2T_1$ の関係があります。密度行列で見ると、非対角成分 $\rho_{01}, \rho_{10} \to 0$ となります。

脱分極エラーとの違い

性質	脱分極エラー	T1/T2 緩和
対称性	$|0\rangle$ と $|1\rangle$ を対称に扱う	$|0\rangle$ が安定、$|1\rangle$ が不安定
収束先	対角成分 $\to (\frac{1}{2}, \frac{1}{2})$	対角成分 $\to (1, 0)$

本文で脱分極エラーを採用した理由は、ゲートエラーの主要成分として妥当な近似であること、対称性により解析が簡潔になること、そして回路実行時間が T1/T2 より十分短い場合は T1/T2 の影響が相対的に小さいことです。より精密なシミュレーションでは、ゲートエラー（脱分極）と待機時間中のデコヒーレンス（T1/T2）を組み合わせてモデル化します。

FTQC の実現

ここまで見た通り、誤り訂正のない NISQ デバイスでは、回路が深い Shor のアルゴリズムの実行は非常に難しい。Shor のアルゴリズムを実行するには、誤り訂正を備えた FTQC（誤り耐性量子計算）が必要である。

例えば、RSA-2048 を破るには現状約 4,100〜6,189 個の論理量子ビットが必要（構成方式により異なる：Beauregard (2003) [3] の $2n+3$ 構成で約 4,100、Gidney & Ekerå (2021) [4] の構成で約 6,189）。しかし、2026 年 4 月現在で実証されている論理量子ビット数は最大 100 弱（[[8,3,2]] 符号を含む構成で 48 論理 qubit [7] の後続として、QuEra が距離 d=4 で最大 96 個の論理 qubit 同時稼働を実証 [7a]。さらに 2026 年 3 月には Quantinuum Helios が Iceberg 符号で 94 個の error-detected logical qubits / 48 個の error-corrected logical qubits を同時稼働させ、物理: 論理 ≈ 2:1 の記録を更新している）であり、2 桁近いギャップがある。

さらに、「100 個弱の論理量子ビットが見えてきた」ことは「その上で任意の量子回路を自由に走らせられる」ことを意味しない。FTQC の実現には、以下のような技術的課題が残っている:

- **物理量子ビットのスケールアップ**
  1 つの論理量子ビットを構成するのに表面符号では $\sim d^2$ 個の物理量子ビットが必要であり、RSA-2048 を破るには数百万〜数千万個の物理量子ビットが必要と見積もられている（Gidney & Ekerå (2021) [4] で約 2,000 万、Gidney (2025, preprint) [4a] では 100 万未満と、見積りは継続的に更新されている）
- **マジックステート蒸留のコスト**

Shor のアルゴリズムで大量に必要な非クリフォードゲート（T ゲート）を誤り訂正下で実装するには、マジックステート蒸留という高コストな手順が必要であり、FTQC のリソースの大部分を占める [11][12]

- **リアルタイムデコーディング**
  誤り訂正のシンドローム測定結果を、次のゲート操作までにリアルタイムで処理しフィードバックする必要がある [13]
- **論理量子ビット上での安定した回路実行**
  論理量子ビットを「作れた」ことと、その上で Shor のアルゴリズムのような大規模な回路を長時間安定して実行できることは別の問題である

注意：見積りは固定値ではない
上記の数値は、回路最適化・誤り訂正設計・ハードウェア前提の進歩によって変動します。同研究グループによる継続的な最適化により、より少ないリソースでの実現を目指す研究も進んでいる [4a]。ただし、誤り訂正が前提である点は変わりません。

いつ頃 RSA-2048 を破れる量子コンピュータが実現するのかの参考情報として、量子業界の主要企業のロードマップを以下に整理する（2026 年 4 月時点）。

IBM

- **2029 年（目標）**：200 論理 qubit（Starling）。100M ゲート規模の回路実行を掲げる
- **2033 年以降（目標）**：最大 2,000 論理 qubit（Blue Jay）。1B ゲート規模を掲げる

QuEra

- **2026 年（目標）**：100 論理 qubit。10,000 物理 qubit 規模で、深い論理回路を想定
- **2025 年 11 月（実証）**：最大 96 論理 qubit（符号距離 d=4）。最大 448 原子の論理プロセッサで実験 [7a]

Google

- **2024 年 12 月（実証）**：below-threshold QEC。Willow にて、符号距離を上げると論理エラー率が改善することを実証 [13]

Quantinuum

- **2029 年（目標）**：“Apollo” にて “universal, fully fault-tolerant” 量子計算を目指す。何百の論理 qubit・何千の物理 qubit 規模を想定（2024 年に 12 論理 qubit の実証も達成）

公開されているロードマップを基準にすると、たとえば IBM は 2033+ で最大 2,000 論理 qubit 規模（Blue Jay）を掲げているが、RSA-2048 の推定必要量（構成方式により異なり、Beauregard 系で約 4,100、Gidney & Ekerå 系で約 6,189 論理 qubit）には届かない。

コラム：符号距離とは
量子誤り訂正符号の符号距離（code distance）d は、その符号がどれだけのエラーを訂正でき

るかを表す指標です。符号距離 d の符号は、最大 $\lfloor (d-1)/2 \rfloor$ 個のエラーを訂正できます。例えば $d=3$ なら 1 個、$d=5$ なら 2 個のエラーを訂正可能です。
符号距離を上げるには、より多くの物理 qubit が必要になります。代表的な surface code では、符号距離 d の論理 qubit を構成するのに約 d^2 個の物理 qubit を使います。
重要なのは、物理 qubit のエラー率が一定の閾値より低い場合、符号距離を上げると論理エラー率が指数的に改善することです。Google Willow の成果は、この閾値以下での動作（below-threshold QEC）を実証し、$d=3 \to 5 \to 7$ とスケールアップしたときに論理エラー率が約 2 倍ずつ改善することを示した点にあります。

公的機関による FTQC への移行期限

FTQC が実現すれば、Shor のアルゴリズムにより RSA 暗号は破られる。そのため、各国政府は現在使われている RSA/ECC 暗号をポスト量子暗号（PQC）に移行するよう求めている。

FTQC の実現は、主要企業のロードマップから推測して 2030 年代半ば以降となる見込みであるが、政府が設定している移行期限はそれより早い。

コラム：鍵長と NIST 安全強度（古典方式）

以下は NIST が定める安全強度（security strength）に基づく RSA 鍵長の位置づけです。

RSA 鍵長	安全強度	運用ステータス（古典方式）
RSA-2048	112 ビット	2030 年 12 月まで acceptable（新規利用可）、2031 年 1 月以降 deprecated（新規用途では非推奨）
RSA-3072	128 ビット	acceptable（RSA-2048 が deprecated になる 2031 年以降、署名生成等の新規保護用途では事実上必要）
RSA-4096	約 140 ビット（NIST 規格には明示されず、外挿値）	acceptable（長期保存データ向け）

安全強度は対称鍵換算での安全性の目安であり、NIST の規格（SP 800-56B Rev.2 / SP 800-57）に基づきます。運用ステータス（acceptable / deprecated 等）は NIST の運用ガイドライン（SP 800-131A Rev.3）の整理に基づきますが、同文書は 2024 年 10 月公開の IPD（Initial Public Draft）であり、最終確定版ではありません。なお、SP 800-131A は署名の「生成」と「検証」でステータスを分けています。たとえば RSA-2048 が deprecated になった後でも、過去に RSA-2048 で生成された署名を検証することは legacy use として引き続き認められます。

NIST（米国標準技術研究所）の指針 [8]

NIST IR 8547（2024 年 11 月・Initial Public Draft）では、量子脆弱な公開鍵暗号の移行スケジュールを security strength ベースで示している。

年	NIST（IR 8547 ipd）の表現	対象のイメージ
2031 年 1 月 1 日以降	112-bit security 相当の量子脆弱な公開鍵暗号を deprecated	RSA-2048 級など
2036 年 1 月 1 日以降	量子脆弱な公開鍵暗号を disallowed（≥ 128-bit security も含む）	ECDH/ECDSA P-256 級も含めて移行完了が必要

NIST は 2024 年 8 月に以下の PQC 標準を発表しており、これらへの移行が求められている [5]。

- **ML-KEM**（FIPS 203）：鍵共有（格子ベース）
- **ML-DSA**（FIPS 204）：署名（格子ベース）
- **SLH-DSA**（FIPS 205）：署名（ハッシュベース）

これらは Shor アルゴリズムが解ける因数分解・離散対数問題ではなく、格子問題やハッシュ問題に基づいている。Shor アルゴリズムでは解けず、現時点で効率的な量子アルゴリズムも知られていない。

さらに、NIST は 2025 年 3 月に HQC を追加の標準化対象として選定した [15]（ML-KEM の代替としての多様性確保）。また、FIPS 206（FN-DSA / FALCON）も策定中であり、ML-DSA より署名サイズが小さい選択肢として位置づけられている。KEM の利用ガイダンスとして SP 800-227 [14] も 2025 年 9 月に Final となった。PQC 標準は今後も拡充・改訂が続く見込みである。

NSA CNSA 2.0（米国国家安全保障局）[9], [9a]

NSA は国家安全保障システム（NSS）向けに、より具体的な期限を設定している。

年	NSS（CNSA 2.0: 2022 年 / CNSSP 15: 2025 年）要求
2025-12-31	それ以前は移行要件を強制しない
2027-01-01	新規調達は CNSA 2.0 準拠が必須（原則）
2030-12-31	非対応の機器・サービスはフェーズアウト
2031-12-31	CNSA 2.0 の使用が原則義務化

米ホワイトハウス / OMB（NSM-10 / M-23-02）

2035 年までの移行目標は、CNSA 2.0 / CNSSP 15 ではなく、米ホワイトハウスの **National Security Memorandum 10 (NSM-10)**（2022 年）および **OMB M-23-02**（2022 年）に基づく。M-23-02 は NSS を**除く**連邦機関を対象に、2035 年までに量子リスクを可能な限り低減すること（“mitigating as much of the quantum risk as is feasible by 2035”）を目標として掲げている。

補足：NIST と NSA の採用アルゴリズムの違い

NSA は NSS 向けに ML-KEM-1024、ML-DSA-87 を指定している。一方、以下のアルゴリズムは NSS では承認されていない：

- **SLH-DSA**（FIPS 205）：NIST は標準化したが、NSA は CNSA 2.0 に含めていない
- **FN-DSA**（FIPS 206、策定中）：実装ミスの影響が大きいとして、CNSA 2.0 に追加する予定はないと明言

代わりに、NSA はソフトウェア/ファームウェア署名用途として **LMS/XMSS**（SP 800-208）を推奨している（ただし HSS/XMSSMT は NSS では不可）。

Harvest Now, Decrypt Later への懸念

量子コンピュータが RSA-2048 を破れるようになるのは 2030 年代半ば以降と予測されているのに、政府が 2031 年以降の移行を求めている理由は「**Harvest Now, Decrypt Later**（直訳すると、今収穫し、後で復号）」攻撃への懸念である。

これは、現時点で暗号化された通信を傍受・保存しておき、将来 FTQC が実現した時点で復号するという攻撃である。国家機密や医療情報など、10 年、20 年先も秘匿が必要なデータは多く存在する。2024 年 8 月に NIST が PQC 標準を発表したことで、各組織での移行が始まっている。

3.6　まとめ

本章では、Shor のアルゴリズムについて以下を解説した。

位数発見問題

Shor のアルゴリズムは素因数分解を直接解くのではなく、「位数発見」という別の問題に帰着させる。$a^r \equiv 1 \pmod{n}$ を満たす最小の r（位数）が分かれば、$\gcd(a^{r/2} \pm 1, n)$ から素因数が求まる。この帰着は純粋に数論の話であり、量子計算とは無関係である。

量子位相推定（QPE）

位数を効率的に求めるために量子位相推定を用いる。QPE は、ユニタリ演算子 U の固有値 $e^{2\pi i\varphi}$ から位相 φ を読み取る汎用的な量子アルゴリズムである。制御ビットにアダマールゲートを適用して重ね合わせを作り、制御ユニタリ演算で位相情報を埋め込み、逆量子フーリエ変換で位相を測定可能な形に変換する。

Shor のアルゴリズムの手順

Shor のアルゴリズムは、QPE を位数発見に特化させた形である。モジュラー乗算ユニタリ $U_{a,n}|x\rangle = |ax \bmod n\rangle$ の固有値から位相 $\varphi = s/r$ を読み取り、連分数展開により位数 r を求める。測定確率は $z = s \cdot 2^M/r$ 付近にピークが立ち、正しい z を高確率で得られる。

実用性の分析

計算量について、$L = \log_2 n$（n のビット長）として、古典的な一般数体ふるい法が準指数時間 $O\left(\exp\left(c \cdot (\ln n)^{1/3}(\ln\ln n)^{2/3}\right)\right)$（自然対数 ln で表記。02 章と統一）であるのに対し、Shor のアルゴリズムは多項式時間 $O(L^3)$ で動作する。この準指数時間から多項式時間への超多項式な高速化が、RSA 暗号への脅威となる。

ただし現状では実行できない。誤り訂正のない NISQ デバイスではエラーが蓄積し、誤り訂正を備えた FTQC が必要である。RSA-2048 には構成方式により約 4,100〜6,189 個の論理量子ビット（Beauregard 系で約 4,100、Gidney & Ekerå 系で約 6,189）が必要だが、現状は最大 100 弱であり、FTQC の実現は 2030 年代半ば以降と見られている。

FTQC が実現すれば RSA/ECC 暗号は破られるため、各国政府はポスト量子暗号（PQC）への移行を求めている。NIST は 2024 年 8 月に ML-KEM、ML-DSA、SLH-DSA を標準化し、移行が始まっている。

本章を通して、「量子コンピュータが RSA を破る」とは何を意味するのかが明確になれば幸いである。

注記：本章の 3.5.6 節で引用した数値・見積り・ロードマップは、2026 年 4 月時点で公開されている文献に基づいています。量子コンピューティングは急速に発展する分野であり、これらの数値は今後の技術進展により変動し得ます。最新の状況については、各参考文献の原典を確認してください。

3.7　参考文献

- [1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997. (初出は FOCS 1994; 改訂版が 1997 SIAM J. Comput.)
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th Anniversary Edition, 2010.
- [3] S. Beauregard, “Circuit for Shor’s algorithm using 2n+3 qubits,” *Quantum Information & Computation*, vol. 3, no. 2, pp. 175–185, 2003.
- [4] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, p. 433, 2021. https://doi.org/10.22331/q-2021-04-15-433
- [4a] C. Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits,” *arXiv:2505.15917*, 2025.（surface code 前提での最適化見積り。物理量子ビット数を 2,000 万から 100 万未満へと改善）
- [5] NIST, “Post-Quantum Cryptography Standardization,” FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), August 2024. https://csrc.nist.gov/projects/post-quantum-cryptography
- [6] M. Ekerå, “On the success probability of quantum order finding,” *arXiv:2201.07791*, 2022.
- [7] D. Bluvstein et al., “Logical quantum processor based on reconfigurable atom arrays,” *Nature*, vol. 626, pp. 58–65, 2024. https://doi.org/10.1038/s41586-023-06927-3（[[8,3,2]] 符号を用いた 48 論理 qubit 同時稼働と、表面符号（d=3〜7）でのスケーリング実証）
- [7a] D. Bluvstein et al., “A fault-tolerant neutral-atom architecture for universal quantum computation,” *Nature*, vol. 649, pp. 39–46, 2026 (published online 2025-11-10). doi:10.1038/s41586-025-09848-5（距離 d=4 での最大 96 論理 qubit 同時稼働の実証。[7] の後続研究）
- [8] NIST, “Transition to Post-Quantum Cryptography Standards,” NIST IR 8547 (Initial Public Draft), November 2024. https://csrc.nist.gov/pubs/ir/8547/ipd

- [9] NSA, “Announcing the Commercial National Security Algorithm Suite 2.0,” September 2022. https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF
- [9a] CNSS, “Use of Public Standards for Secure Information Sharing,” CNSS Policy No. 15, March 2025.
- [10] M. Kjaergaard et al., “Superconducting Qubits: Current State of Play,” *Annual Review of Condensed Matter Physics*, vol. 11, pp. 369–395, 2020. https://doi.org/10.1146/annurev-conmatphys-031119-050605（超伝導量子ビットのエラー率等の代表的数値として参照）
- [11] S. Bravyi and A. Kitaev, “Universal quantum computation with ideal Clifford gates and noisy ancillas,” *Physical Review A*, vol. 71, 022316, 2005. https://arxiv.org/abs/quant-ph/0403025（マジックステート蒸留の提案論文）
- [12] C. Gidney and A. G. Fowler, “Efficient magic state factories with a catalyzed $|\mathrm{CCZ}\rangle$ to $2|\mathrm{T}\rangle$ transformation,” *Quantum*, vol. 3, 135, 2019. https://quantum-journal.org/papers/q-2019-04-30-135/（arXiv:1812.01238。マジックステート蒸留が FTQC リソースの大部分を占めることの見積もり）
- [13] Google Quantum AI, “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, pp. 920–926, 2025 (online 2024-12-09). https://www.nature.com/articles/s41586-024-08449-y（表面符号でのリアルタイムデコーディングの実証）
- [14] NIST, “Recommendations for Key-Encapsulation Mechanisms,” SP 800-227 (Final), September 2025. https://csrc.nist.gov/pubs/sp/800/227/final（KEM の一般的なガイドライン）
- [15] NIST, “Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process,” NIST IR 8545, 2025. https://csrc.nist.gov/pubs/ir/8545/final（ML-KEM 等に加え、符号ベース KEM である HQC を第 5 の PQC 標準として追加選定）